PatrolBot: AI Surveillance System for Real-Time Threat Detection and Situational Awareness in Public and Private Environments

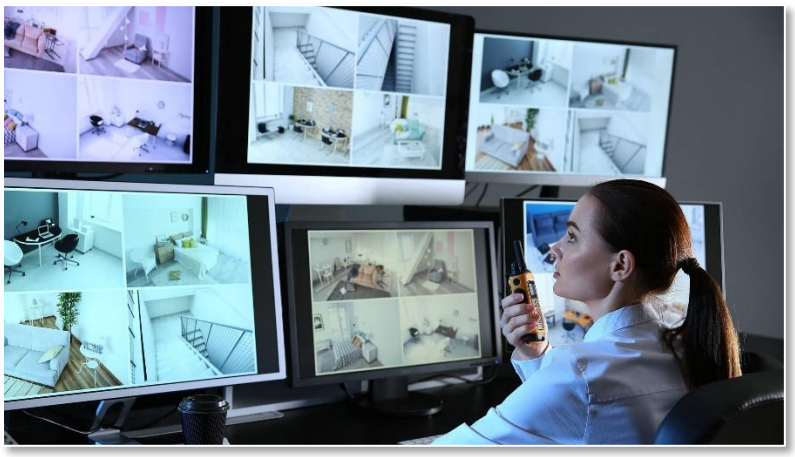
Abstract

Surveillance systems are essential for maintaining security in public and private spaces, yet traditional solutions often suffer from limited coverage, manual monitoring, and frequent false alarms, reducing their effectiveness in detecting threats. Rising urban crime rates and complex environments necessitate intelligent monitoring approaches capable of accurately detecting both objects and abnormal human behaviors in real-time. This project introduces an AI-driven surveillance framework that integrates Swin Transformer (Swin-T) for object and threat detection with Pose-based Graph Convolutional Networks (Pose-GCN) for activity recognition. Swin-T captures hierarchical spatial relationships in complex scenes, accurately detecting multiple or overlapping objects such as weapons, unauthorized entry, or suspicious packages. Pose-GCN leverages human skeleton keypoints to analyze motion patterns, enabling precise identification of abnormal behaviors including loitering, fighting, or unusual movement trajectories. Upon detection of threats or abnormal actions, the system triggers immediate alerts via SMS and email notifications to designated personnel, while maintaining logs for post-incident analysis and audit purposes. By combining advanced spatial analysis with robust behavioral modeling, this surveillance system provides a proactive, accurate, and scalable solution for real-time security monitoring. The integration of Swin-T and Pose-GCN ensures high detection precision even in crowded or complex environments, significantly reducing false alarms and enhancing situational awareness for rapid response.

INTRODUCTION

1.1. OVERVIEW

Surveillance systems are an essential part of securing your home or business. These systems can range from wireless home security cameras to sophisticated alarm systems that notify law enforcement at the first sign of trouble. The presence of security cameras can serve as a deterrent to would-be thieves, while hidden cameras can protect discretely. The term Surveillance means the act of observing and monitoring someone or something. So, a remote video surveillance system is simply a way to observe and monitor an area using video cameras. A video surveillance system is a network of cameras, monitors/display units, and recorders. Cameras can be analog or digital, with various features to explore, such as resolution, frame rate, color type, and more. Whether applied inside or outside the building, it operates 24/7, designed only for recording movement when necessary. Surveillance cameras may be in plain sight or hidden from view. The camera's purpose is to deter improper behavior, and the video footage can also serve as evidence for later review by security staff or law enforcement.



Depending on needs, many different video surveillance systems are available, such as live monitoring, remote access via an IP system, and Digital Video Recorders (DVR) for recording footage. The majority of video surveillance systems are designed to be secure, preventing signal broadcasting to unauthorized entities. Only individuals with the requisite authorization can view

the recorded content. Nonetheless, an administrator with the appropriate credentials who oversees the live footage can grant access to others.

1.2. AIM AND OBJECTIVE

Aim

The aim of this project is to develop an intelligent AI-powered surveillance system capable of real-time threat detection, abnormal behavior recognition, and automated alert generation to enhance situational awareness, improve response times, and strengthen overall security in public and private environments. The system is designed to operate reliably even in crowded or complex scenes, reducing human monitoring effort and ensuring proactive security management.

Objectives

- To detect threats and suspicious objects such as weapons, intrusions, or unattended packages in real time.
- To accurately identify abnormal human behaviors including loitering, fighting, trespassing, or unusual movement patterns.
- To send instant alerts via SMS, email, or dashboard notifications to authorized security personnel.
- To minimize false alarms and improve detection precision using advanced AI models like Swin Transformer and Pose-GCN.
- To maintain detailed logs of all detected events for auditing, forensic analysis, and trend evaluation.
- To enhance situational awareness across diverse environments, providing operators with clear visualizations and actionable insights.
- To support quick and informed security decision-making, enabling rapid intervention and improved safety management.

1.3. SCOPE OF THE PROJECT

The project is designed to enhance security monitoring and threat detection in public and private environments through intelligent, AI-driven surveillance. By integrating advanced deep learning models such as Swin Transformer for object detection and Pose-GCN for abnormal behavior recognition, the system ensures accurate, real-time monitoring with reduced false alarms and improved situational awareness.

The system's scope extends to the following key areas:

- **Real-Time Object and Threat Detection:** Detects suspicious objects such as weapons, unauthorized access, or abandoned packages using advanced vision-based deep learning models.
- **Abnormal Behavior Recognition:** Identifies unusual human activities such as loitering, fighting, or suspicious movement patterns through pose-based activity analysis.
- **Automated Alert and Notification System:** Sends instant SMS and email alerts to authorized personnel when potential threats are detected for rapid response.
- **Live Monitoring and Centralized Dashboard:** Provides a user-friendly interface for security personnel to monitor camera feeds, view alerts, and access system logs in real time.
- **Event Logging and Incident Analysis:** Maintains secure records of detected events for post-incident review, auditing, and evidence management.
- **Multi-Camera and Crowded Environment Handling:** Supports monitoring of multiple CCTV feeds simultaneously while maintaining detection accuracy in complex or crowded scenes.
- **Decision Support and Threat Assessment:** Utilizes analytical models to classify threat severity and assist security teams in prioritizing responses.
- **Scalability and Future Expansion:** Designed to support integration with drones, edge devices, emergency response systems, and cloud-based centralized monitoring platforms.

This project aims to transform traditional surveillance methods into a proactive, automated, and intelligent security solution that improves response time, operational efficiency, and overall public and private safety management.

1.4. PROBLEM STATEMENT

The problem statement of the project is that in today's dynamic public and private environments, maintaining effective security has become increasingly challenging due to rising crime rates, unauthorized access, and abnormal activities. Traditional surveillance systems relying on manual monitoring and static cameras often suffer from limited coverage, delayed response, and frequent false alarms, making it difficult for security personnel to accurately detect threats in real time. These systems lack automated threat recognition, intelligent behavior analysis, and proactive alert mechanisms, forcing operators to manually monitor multiple camera feeds, which increases the risk of missed incidents, slow intervention, and compromised safety. To overcome these challenges, there is a need for a real-time, intelligent, and automated surveillance system capable of accurately detecting threats such as weapons, intrusions, and abnormal human behaviors, notifying security personnel instantly, and maintaining detailed event logs for auditing and investigation. By leveraging Swin Transformer–based object detection, Pose-GCN behavior recognition, and Decision Tree–based threat assessment, the proposed project provides a reliable, efficient, and scalable solution, enhancing situational awareness, reducing false alarms, and enabling rapid, informed security responses in complex environments.

SYSTEM ANALYSIS

2.1. EXISTING SYSTEM

Traditional surveillance systems primarily rely on basic monitoring techniques and conventional machine learning models for detecting suspicious activities. While these systems provide fundamental security support, they often lack intelligence, automation, and adaptability required for modern complex environments.

- **Manual Monitoring and Intervention**

In conventional surveillance setups, security personnel continuously monitor live CCTV feeds or review recorded footage to identify suspicious activities. Operators manually observe movements, behaviors, and environmental changes. When abnormal activities are detected, they report or respond accordingly. However, continuous monitoring leads to human fatigue, delayed response, and potential oversight of critical incidents.

- **Motion Detection Techniques**

Basic surveillance systems use motion sensors or frame-differencing techniques to detect movement. Any sudden or unusual motion triggers an alert. While simple and cost-effective, motion detection often generates false alarms due to environmental factors such as lighting changes, shadows, animals, or weather conditions. It cannot accurately distinguish between normal and suspicious activities.

- **Support Vector Machines (SVM)**

Support Vector Machines are supervised machine learning algorithms used to classify activities into predefined categories such as normal or abnormal. These systems depend heavily on handcrafted feature extraction from video data. Although SVMs can perform basic classification, they struggle with complex scenes, multiple overlapping objects, and dynamic human behaviors in real-world surveillance environments.

- **Autoencoders for Anomaly Detection**

Autoencoders are unsupervised neural networks that learn patterns of normal activities and detect deviations as anomalies. They reconstruct input frames and flag irregular patterns. While useful

for anomaly detection, autoencoders may fail in crowded or highly dynamic environments where defining “normal behavior” is difficult and continuously changing.

- Heavy reliance on manual monitoring, leading to human error and fatigue.
- High false-alarm rates due to simple motion-based detection methods.
- Limited ability to understand complex human behaviors and contextual information.
- Difficulty in distinguishing between normal variations and real threats.
- Poor scalability in crowded or multi-camera environments.
- High computational cost in some traditional models with slower real-time performance.

2.2. PROPOSED SYSTEM

The proposed system introduces an AI-driven surveillance framework designed to automatically detect threats and abnormal human behaviors in real time. By integrating advanced deep learning models for object detection and pose-based action recognition, the system continuously analyzes live CCTV footage and provides accurate, timely security insights while minimizing false alarms and manual intervention.

- **Intelligent Object Detection**

This module employs the Swin Transformer to accurately identify potential threats such as weapons, suspicious packages, and unauthorized intrusions. Its hierarchical and window-based attention mechanism allows the system to capture fine-grained spatial details, enabling reliable detection even in crowded, low-light, or complex environments.

- **Abnormal Behavior Recognition**

The system uses Pose-based Graph Convolutional Networks (Pose-GCN) to analyze human skeletal keypoints extracted from video frames. By understanding body posture and motion patterns, it effectively detects abnormal behaviors such as loitering, fighting, trespassing, or unusual movement trajectories, improving behavior-level interpretation and reducing unnecessary alerts.

- **Real-Time Alert and Notification System**

Once a potential threat or abnormal activity is confirmed, the system immediately triggers alerts through SMS and email notifications to authorized security personnel. These alerts include critical

details such as location, time, and threat type, enabling rapid response and effective coordination during emergency situations.

- **Event Logging and Monitoring Dashboard**

All detected incidents, alerts, and system decisions are automatically stored in a secure database and displayed on a centralized monitoring dashboard. This module allows security teams to view live feeds, review historical incidents, perform audits, and analyze patterns to support future risk prevention and security planning.

- Protects public and private spaces from security threats and abnormal activities.
- Detects potential incidents early to prevent escalation and damage.
- Reduces reliance on manual surveillance and human intervention.
- Minimizes false alarms caused by simple motion or rule-based systems.
- Enhances real-time awareness of ongoing situations and risks.
- Improves response efficiency during emergencies and critical events.
- Supports investigation and forensic analysis through detailed incident records.
- Lowers long-term operational and security management costs.
- Scales easily across multiple locations and large surveillance networks.

SYSTEM REQUIREMENTS

3.1 HARDWARE REQUIREMENTS

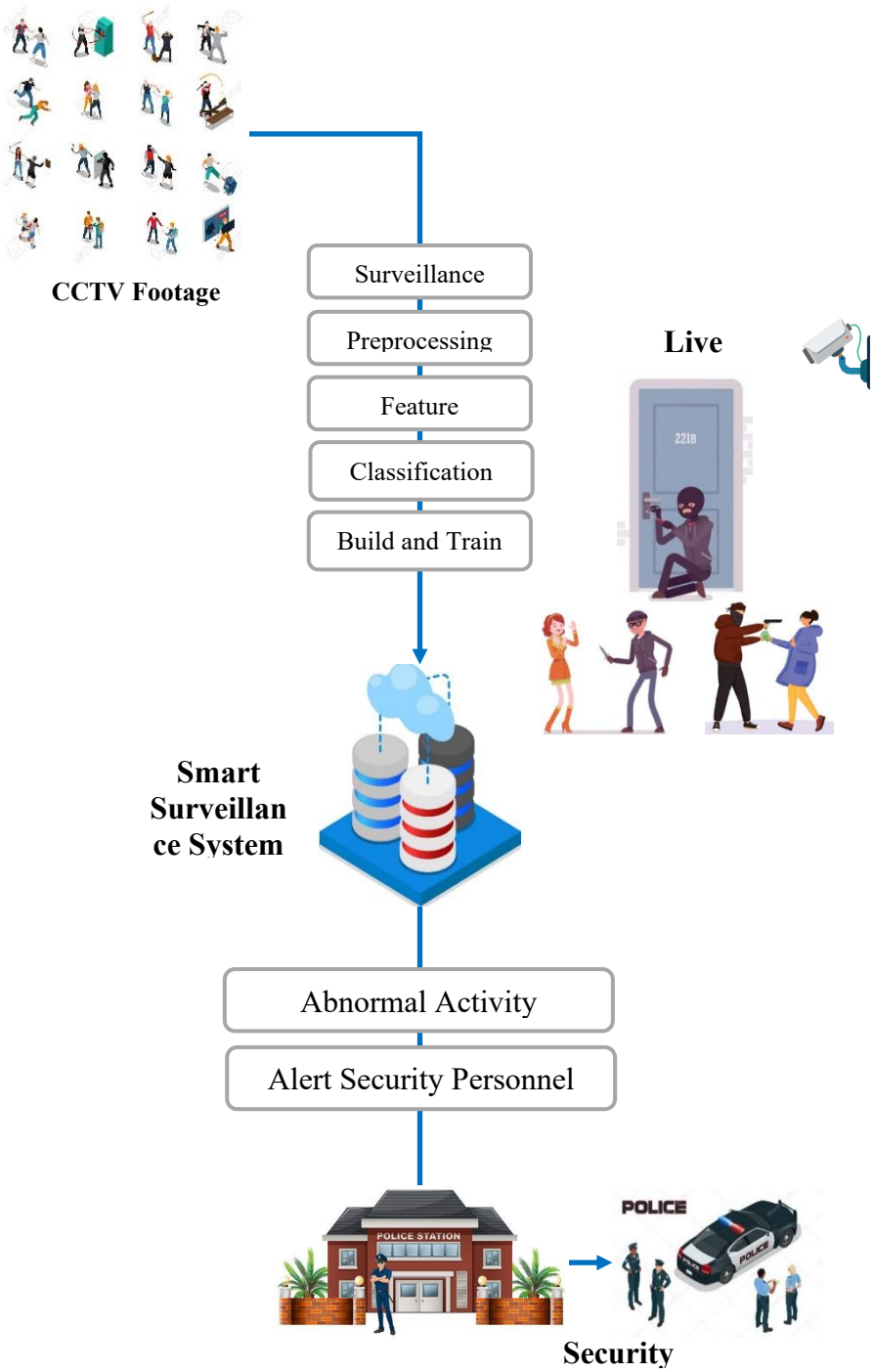
- **Processor** **:** Intel Core i5 / i7 or higher
- **RAM** **:** Minimum 8 GB (16 GB recommended)
- **Storage** **:** 256 GB SSD or higher
- Network Connectivity (LAN / Wi-Fi)

3.2. SOFTWARE REQUIREMENTS

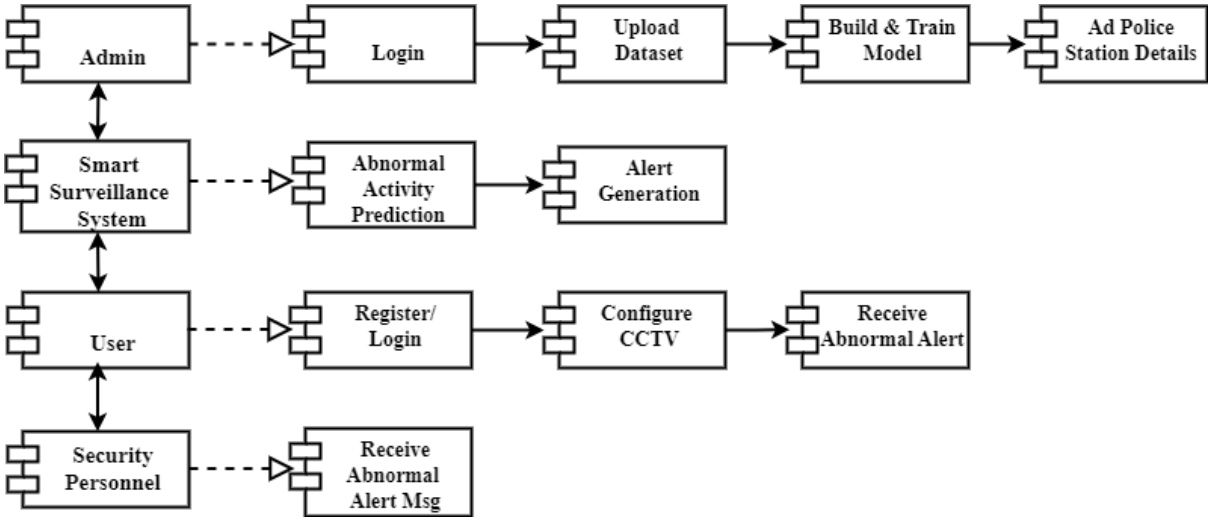
- **Operating System** **:** Windows 10 / Windows 11
- **Programming Language** **:** Python 3.x
- **Frontend** **:** HTML, CSS, Bootstrap
- **Backend Framework** **:** Flask
- **Database** **:** MySQL
- **Server Environment** **:** Wamp Server
- **Libraries** **:** OpenCV, TensorFlow, Keras, NumPy, Pandas, Scikit-learn
- **Notification Services** **:** SMTP (Email), SMS API

SYSTEM DESIGN

4.1. SYSTEM ARCHITECTURE

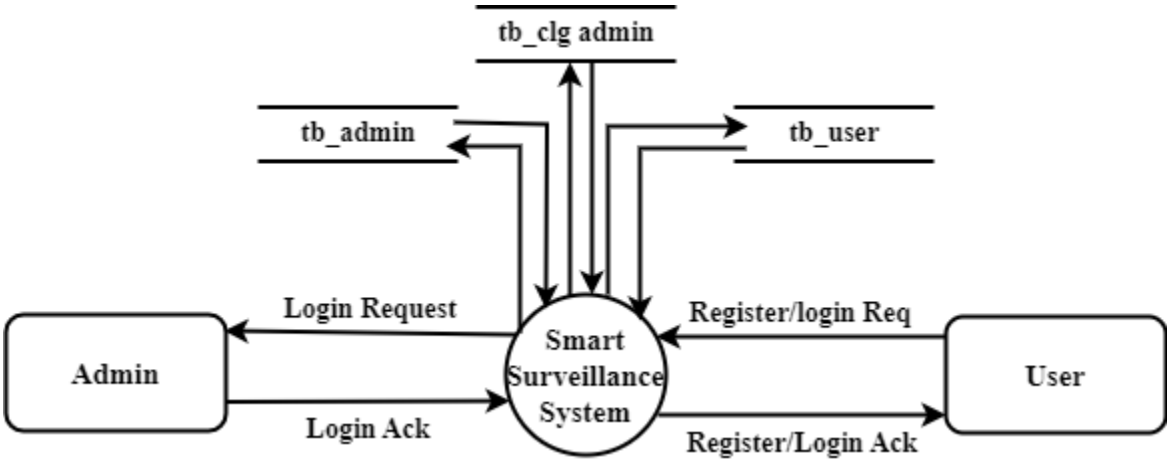


4.1.2. COMPONENT DIAGRAM

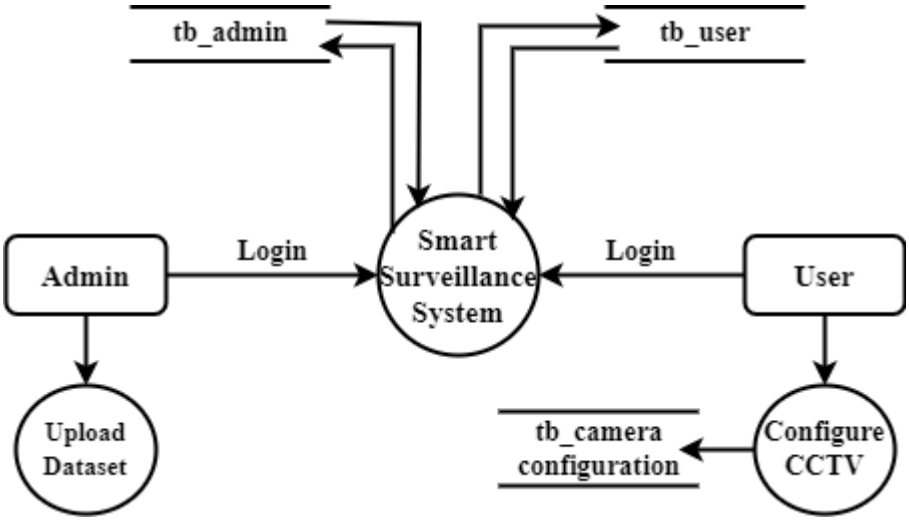


4.2. DATA FLOW DIAGRAM

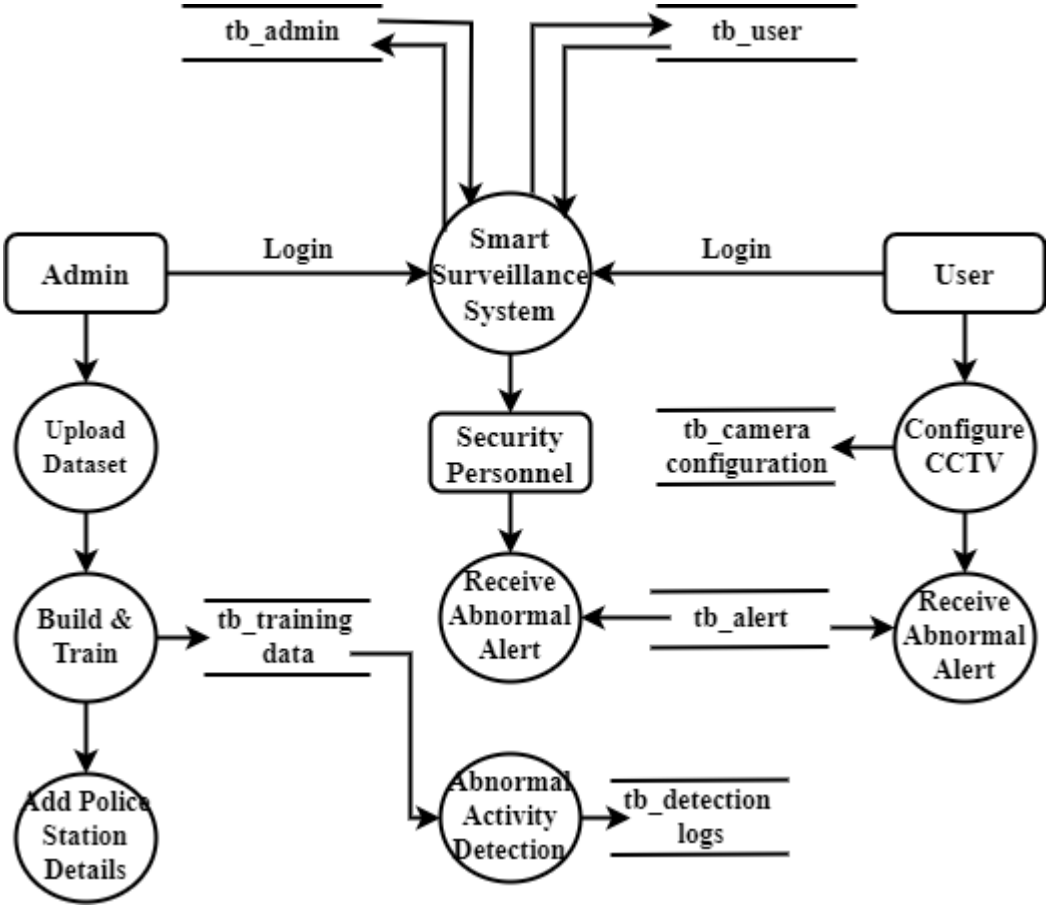
LEVEL 0



LEVEL 1

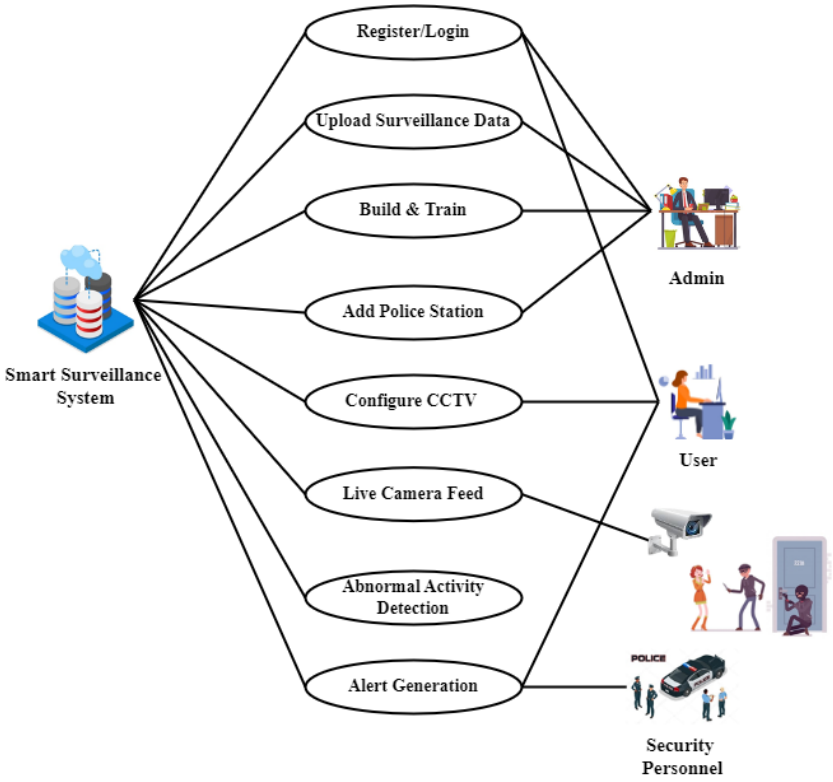


LEVEL 2

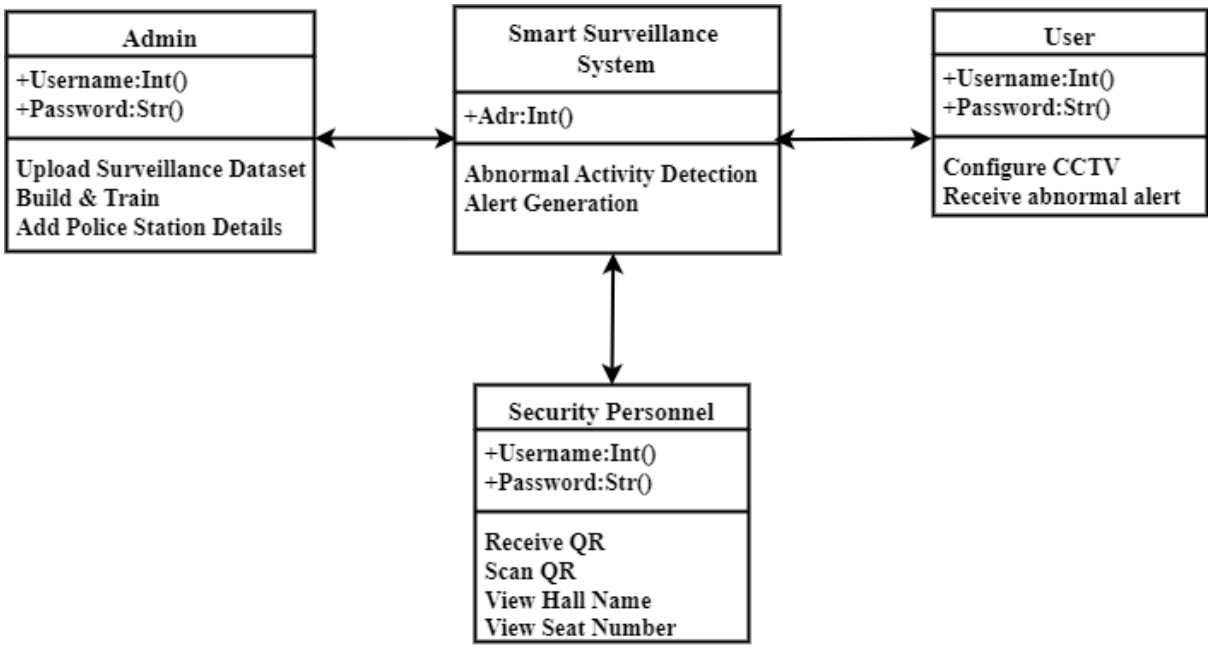


4.3. UML DIAGRAM

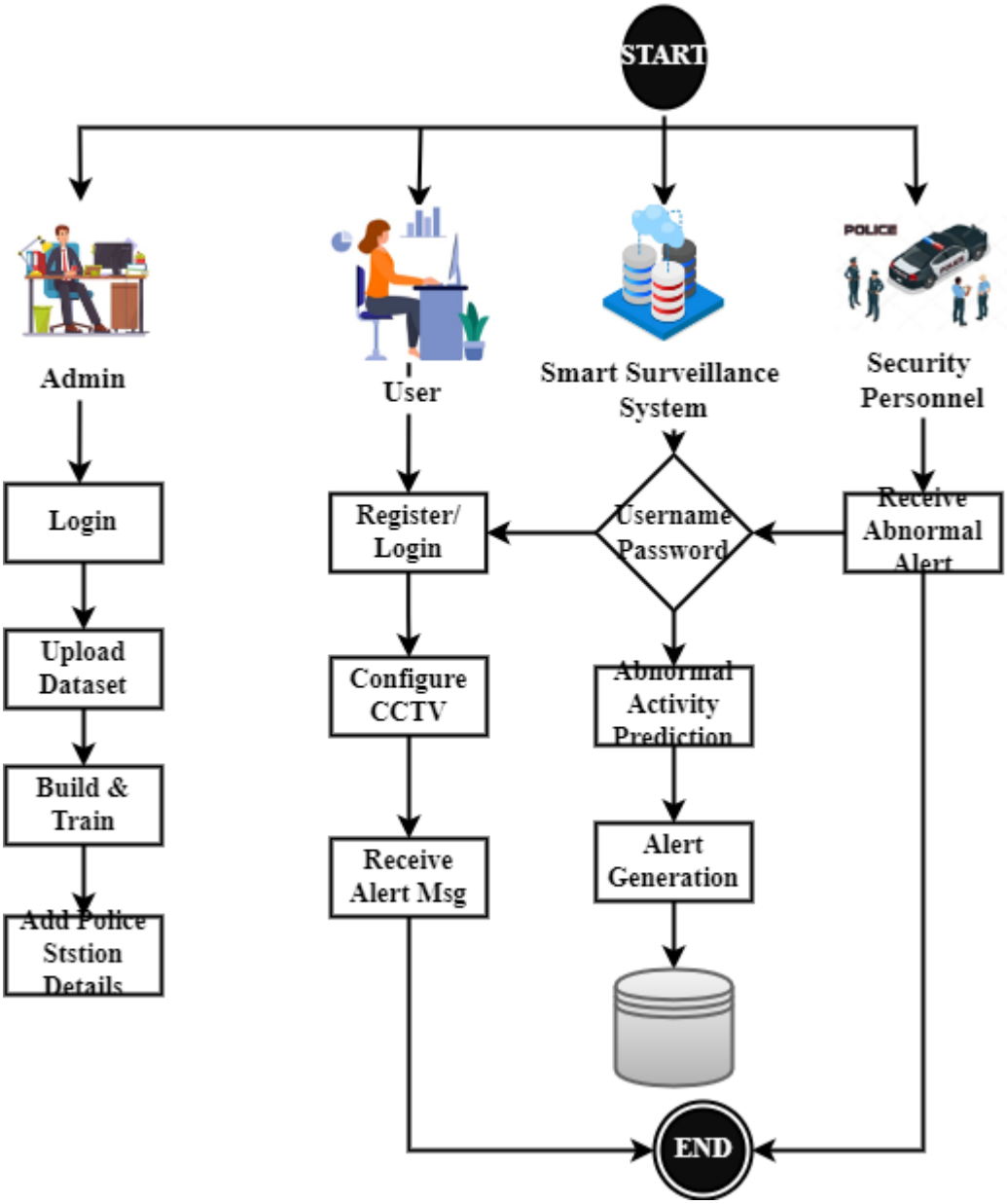
4.3.1. USE CASE



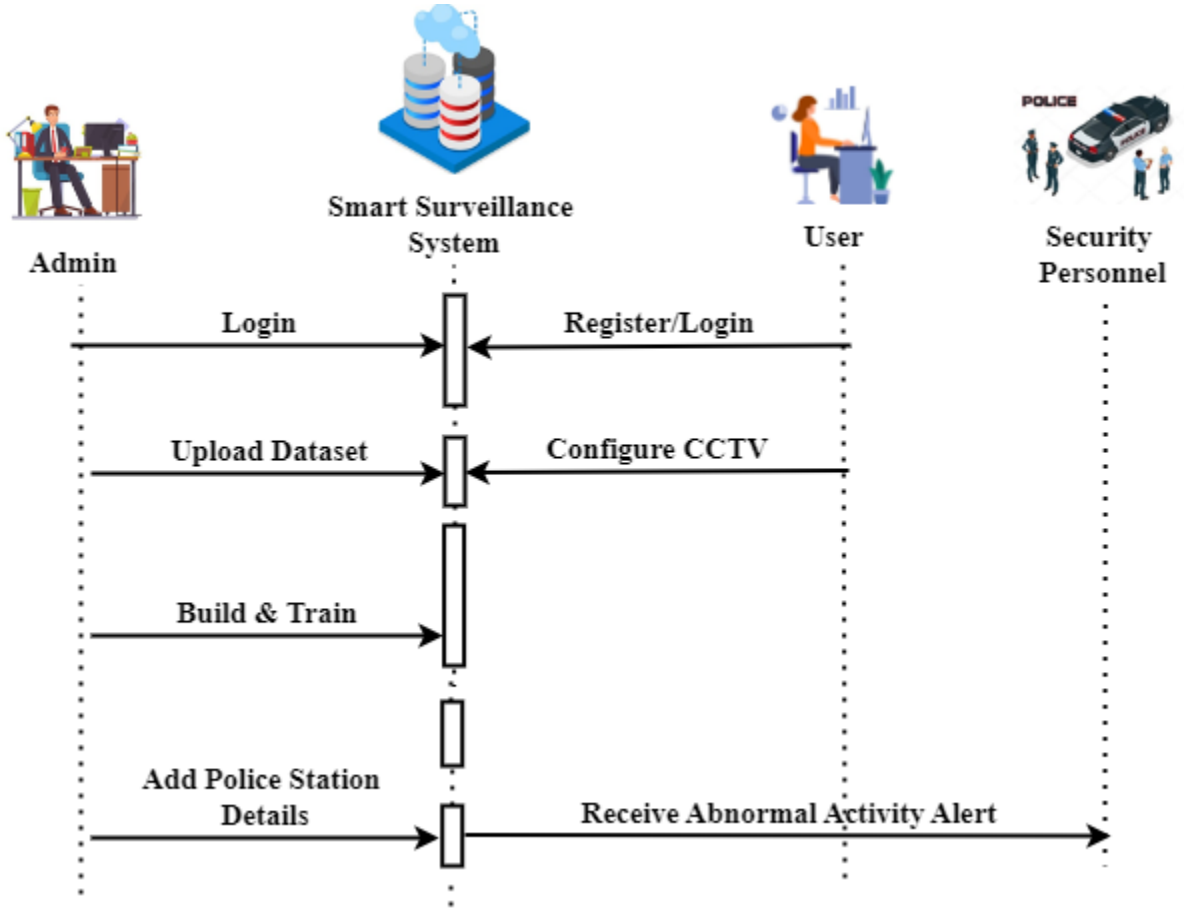
4.3.2. CLASS DIAGRAM



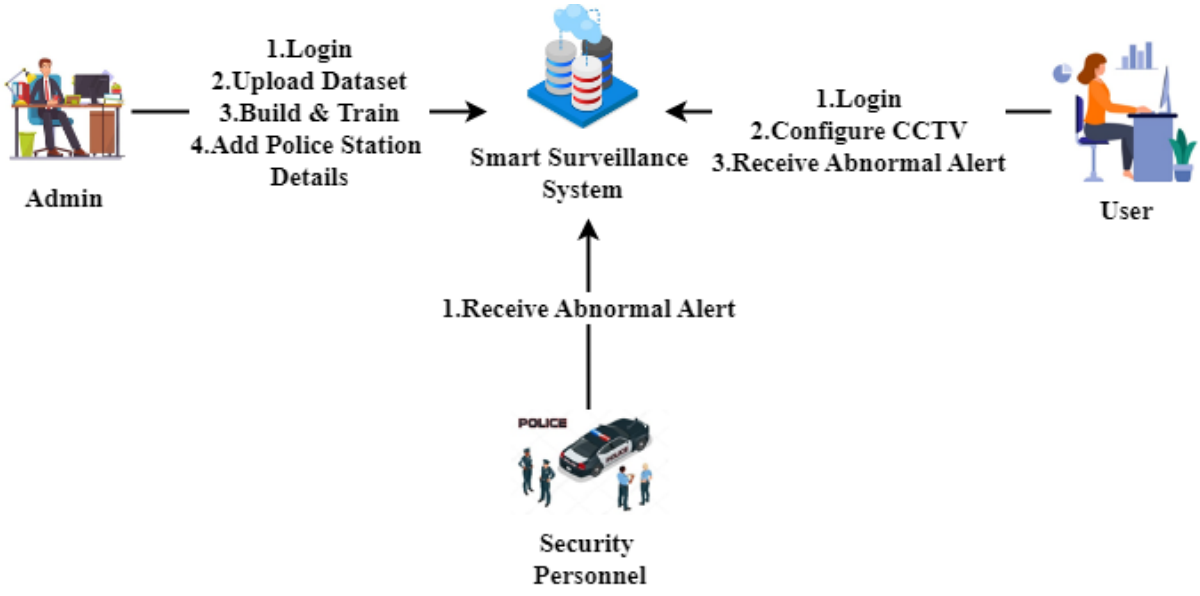
4.3.3. ACTIVITY DIAGRAM



4.3.4. SEQUENCE DIAGRAM



4.3.5. COLLABORATION DIAGRAM



4.4. MODULES DESCRIPTION

1. PatrolBot Dashboard

This module focuses on the design and development of the central control dashboard using Python, Flask, MySQL, Bootstrap, Wamp Server, and necessary project libraries. It provides an intuitive web interface where users can view live CCTV feeds, detected threats, alerts, logs, and system status. Flask handles backend logic and communication with AI models, while MySQL stores user data, alerts, incident logs, and configurations. Bootstrap ensures a responsive and user-friendly UI for desktops and control rooms. The dashboard also includes features such as camera management, alert history, role-based login, and visualization overlays that highlight detected objects and abnormal activities in real time.

2. System Users

2.1 Security Personnel

Security personnel primarily use this system for real-time monitoring. They watch live video streams, receive immediate alerts, and respond to suspicious activities or confirmed threats. Their interface focuses on clear visualization and fast action buttons, enabling rapid intervention.

2.2 Surveillance Operators / Control Room Staff

Operators oversee multiple camera feeds and verify alerts generated by the AI. They analyze visual evidence, coordinate with field security teams, and escalate confirmed threats to higher authorities when required. Their role ensures accuracy and reduces false responses.

2.3 System Administrators

Administrators manage user accounts, permissions, camera configurations, alert thresholds, and system maintenance. They also handle model training, updates, and performance monitoring. Administrators ensure system reliability, security, and correct configuration for operational environments.

2.4 Law Enforcement / Police Authorities

Authorities use the system during investigations. They access incident logs, recorded evidence, timestamps, and alert histories to analyze events. The system supports post-incident review, case reconstruction, and legal documentation.

3. PatrolNet: Build and Train

3.1 Import Dataset

In this sub-module, relevant surveillance datasets are collected and organized. The dataset contains both normal activities and threat scenarios so that the system can learn balanced behavior patterns. Each video sample is labeled for critical objects such as weapons, suspicious packages, and intrusions, as well as behavioral categories such as loitering, aggression, and trespassing. These labeled datasets form the foundation for training the PatrolNet model with realistic real-world variations.

3.2 Pre-processing

During this stage, raw video data is converted into a form suitable for machine learning. Frames are extracted from the video streams, resized to a consistent resolution, and converted to grayscale to reduce computational complexity. A median filter is applied to remove noise without losing important edges. Background subtraction is then used to isolate moving foreground objects from static surroundings. Together, these operations enhance visual clarity, eliminate unwanted distortions, and prepare accurate inputs for further feature learning.

3.3 Feature Extraction

In this sub-module, self-attention mechanisms are applied to the processed frames. The network automatically learns which regions in the image are most important by assigning higher attention weights to relevant features. As a result, objects, human figures, and significant motion patterns become more distinct. This helps the model focus only on meaningful information and ignore unnecessary background content, improving both accuracy and robustness in complex environments.

3.4 PatrolNet Model: Build and Train

This sub-module is responsible for building and training the combined AI architecture. The PatrolNet pipeline integrates the Swin Transformer for precise object and threat detection, while the Pose-GCN network is used for abnormal behavior recognition based on human pose structures. Training is performed using the prepared dataset, allowing the model to learn patterns related to weapons, suspicious packages, intrusions, loitering, fighting, and unusual activities. Hyperparameters are carefully tuned to maintain a balance between detection accuracy and real-time processing performance.

3.5 Deploy Model

After successful training, the PatrolNet model is exported and integrated into the PatrolBot Dashboard. Once deployed, it processes live CCTV streams continuously and generates detection outputs in real time. These predictions are forwarded to the threat-assessment and alert modules, enabling the full system to operate autonomously during real-world surveillance.

4. Threat Assessment & Decision

4.1 Input CCTV Footage

In this sub-module, live video streams from CCTV cameras are captured and fed directly into the system. The continuous input ensures uninterrupted monitoring and enables real-time processing of every frame.

4.2 Pre-processing

The same preprocessing logic used during training is applied during real-time operation to maintain consistency. Each captured frame undergoes conversion, resizing, grayscale transformation, median noise filtering, and background subtraction. This ensures that the model receives clean, standardized data for analysis.

4.3 Feature Extraction

Self-attention is again applied to highlight the most relevant spatial regions and movement cues in the scene. By emphasizing the important features before detection, the system can process complex environments while maintaining high accuracy and speed.

4.4 Object Detection

In this stage, the Swin-Transformer detects suspicious objects and possible threats in the video. It is capable of handling overlapping objects, changing illumination, and crowded scenes. Detected objects are marked with bounding boxes and passed forward for further evaluation.

4.5 Object Tracking

The DeepSORT algorithm assigns unique IDs to detected objects and tracks them across multiple frames. This makes it possible to observe object persistence, direction of movement, and interactions over time. Tracking helps differentiate normal transient objects from persistent threats.

4.6 Abnormal Behavior Recognition

Here, the Pose-GCN model analyzes human skeleton structures and motion sequences. It identifies behaviors such as loitering, aggression, trespassing, or unusual movement trajectories. Because decisions are based on body pose rather than only pixels, behavior detection remains reliable even in visually cluttered environments.

4.7 Threat Detection

In this sub-module, the outputs generated by the trained PatrolNet model — including detected objects from Swin-Transformer and abnormal behaviors identified by Pose-GCN — are passed to a Decision Tree classifier. The Decision Tree evaluates multiple factors such as object type, behavior category, risk score, and time duration to decide whether the situation is safe, suspicious, or a critical threat. Because it uses rules learned during training, the Decision Tree provides transparent and explainable decisions, significantly reducing false alarms while ensuring high-risk events are immediately flagged for response.

5. Monitoring & Visualization

This module acts as the primary interface for real-time situational awareness. All live CCTV feeds are streamed into the dashboard, where the system overlays detection results directly onto the video frames. Detected objects appear with bounding boxes, abnormal human activities are highlighted using pose skeleton overlays, and DeepSORT generates tracking paths that show object movement across frames. Threat levels and alert indicators are also displayed so that operators can immediately recognize high-risk scenarios. In addition to live monitoring, the visualization panel allows users to review historical incidents, filter events based on time or severity, and observe patterns across multiple cameras. By converting raw AI output into clear, meaningful visuals, this module significantly simplifies decision-making for control room staff.

6. Alert and Notification

This module ensures that critical incidents do not go unnoticed. Whenever the Decision Tree confirms a suspicious or high-risk event, the system automatically generates instant SMS and email alerts for designated security personnel and authorities. Each alert message includes the camera location, event timestamp, detected threat type, severity level, and a brief description of the incident. In cases of continuous risk, alerts can be repeated or escalated to higher authorities.

The alert system is designed to minimize human delay, enabling faster response, coordinated communication, and timely intervention even when operators are away from the dashboard.

7. Event Logging & Audit

This module is responsible for maintaining a complete digital record of system activity. Every detection, alert, decision rule applied, and supporting frame is securely stored in the database. The logs include details such as time, camera source, detected object, recognized behavior, risk classification, and user actions taken. These records serve multiple purposes: they assist in investigations, support compliance with surveillance policies, provide evidence for law enforcement, and allow administrators to evaluate system performance. Over time, historical data can be analyzed to identify recurring threat locations, operational weaknesses, or emerging risk trends, enabling continuous improvement of overall security strategy

SOFTWARE DESCRIPTION

5.1. PYTHON 3.8

Python is a general-purpose interpreted, interactive, object-oriented, and high-level programming language. It was created by Guido van Rossum during 1985- 1990. Like Perl, Python source code is also available under the GNU General Public License (GPL). This tutorial gives enough understanding on Python programming language.

Python is a high-level, interpreted, interactive and object-oriented scripting language. Python is designed to be highly readable. It uses English keywords frequently where as other languages use punctuation, and it has fewer syntactical constructions than other languages. Python is a MUST for students and working professionals to become a great Software Engineer specially when they are working in Web Development Domain. Python is currently the most widely used multi-purpose, high-level programming language. Python allows programming in Object-Oriented and Procedural paradigms. Python programs generally are smaller than other programming languages like Java. Programmers have to type relatively less and indentation requirement of the language, makes them readable all the time. Python language is being used by almost all tech-giant companies like – Google, Amazon, Facebook, Instagram, Dropbox, Uber… etc. The biggest strength of Python is huge collection of standard libraries which can be used for the following:

- Machine Learning
- GUI Applications (like Kivy, Tkinter, PyQt etc.)
- Web frameworks like Django (used by YouTube, Instagram, Dropbox)
- Image processing (like OpenCV, Pillow)
- Web scraping (like Scrapy, BeautifulSoup, Selenium)
- Test frameworks
- Scientific computing

- Text processing and many more.

Tensor Flow

TensorFlow is an end-to-end open-source platform for machine learning. It has a comprehensive, flexible ecosystem of tools, libraries, and community resources that lets researchers push the state-of-the-art in ML, and gives developers the ability to easily build and deploy ML-powered applications.

TensorFlow provides a collection of workflows with intuitive, high-level APIs for both beginners and experts to create machine learning models in numerous languages. Developers have the option to deploy models on a number of platforms such as on servers, in the cloud, on mobile and edge devices, in browsers, and on many other JavaScript platforms. This enables developers to go from model building and training to deployment much more easily.

Keras

Keras is a deep learning API written in Python, running on top of the machine learning platform TensorFlow. It was developed with a focus on enabling fast experimentation.

- Allows the same code to run on CPU or on GPU, seamlessly.
- User-friendly API which makes it easy to quickly prototype deep learning models.
- Built-in support for convolutional networks (for computer vision), recurrent networks (for sequence processing), and any combination of both.

- Supports arbitrary network architectures: multi-input or multi-output models, layer sharing, model sharing, etc. This means that Keras is appropriate for building essentially any deep learning model, from a memory network to a neural Turing machine.

Pandas

pandas are a fast, powerful, flexible and easy to use open source data analysis and manipulation tool, built on top of the Python programming language. pandas are a Python package that provides fast, flexible, and expressive data structures designed to make working with "relational" or "labeled" data both easy and intuitive. It aims to be the fundamental high-level building block for doing practical, real world data analysis in Python.

Pandas is mainly used for data analysis and associated manipulation of tabular data in Data frames. Pandas allows importing data from various file formats such as comma-separated values, JSON, Parquet, SQL database tables or queries, and Microsoft Excel. Pandas allows various data manipulation operations such as merging, reshaping, selecting, as well as data cleaning, and data wrangling features. The development of pandas introduced into Python many comparable features of working with Data frames that were established in the R programming language. The panda's library is built upon another library NumPy, which is oriented to efficiently working with arrays instead of the features of working on Data frames.

NumPy

NumPy, which stands for Numerical Python, is a library consisting of multidimensional array objects and a collection of routines for processing those arrays. Using NumPy, mathematical and logical operations on arrays can be performed.

NumPy is a general-purpose array-processing package. It provides a high-performance multidimensional array object, and tools for working with these arrays.

Matplotlib

Matplotlib is a comprehensive library for creating static, animated, and interactive visualizations in Python. Matplotlib makes easy things easy and hard things possible.

Matplotlib is a plotting library for the Python programming language and its numerical mathematics extension NumPy. It provides an object-oriented API for embedding plots into applications using general-purpose GUI toolkits like Tkinter, wxPython, Qt, or GTK.

Scikit Learn

scikit-learn is a Python module for machine learning built on top of SciPy and is distributed under the 3-Clause BSD license.

Scikit-learn (formerly scikits. learn and also known as sklearn) is a free software machine learning library for the Python programming language. It features various classification, regression and clustering algorithms including support-vector machines, random forests, gradient boosting, k-means and DBSCAN, and is designed to interoperate with the Python numerical and scientific libraries NumPy and SciPy.

Pillow

Pillow is the friendly PIL fork by Alex Clark and Contributors. PIL is the Python Imaging Library by Fredrik Lundh and Contributors.

Python pillow library is used to image class within it to show the image. The image modules that belong to the pillow package have a few inbuilt functions such as load images or create new images, etc.

OpenCV

OpenCV is an open-source library for the computer vision. It provides the facility to the machine to recognize the faces or objects.

In OpenCV, the CV is an abbreviation form of a computer vision, which is defined as a field of study that helps computers to understand the content of the digital images such as photographs and videos.

5.1.1 SWIN TRANSFORMER

The Swin Transformer (Shifted Window Transformer) is a type of vision transformer model that processes images by dividing them into small, non-overlapping windows and computes self-attention within these localized regions. Unlike standard vision transformers which use global attention, Swin Transformer introduces a "shifted window" technique. This allows neighboring windows to interact with each other in subsequent layers, efficiently capturing both local and global features in an image.

5.1.1.Swin Transformer

The Swin Transformer's architecture is built on a combination of hierarchical design and window-based self-attention for efficient working and feature extraction.

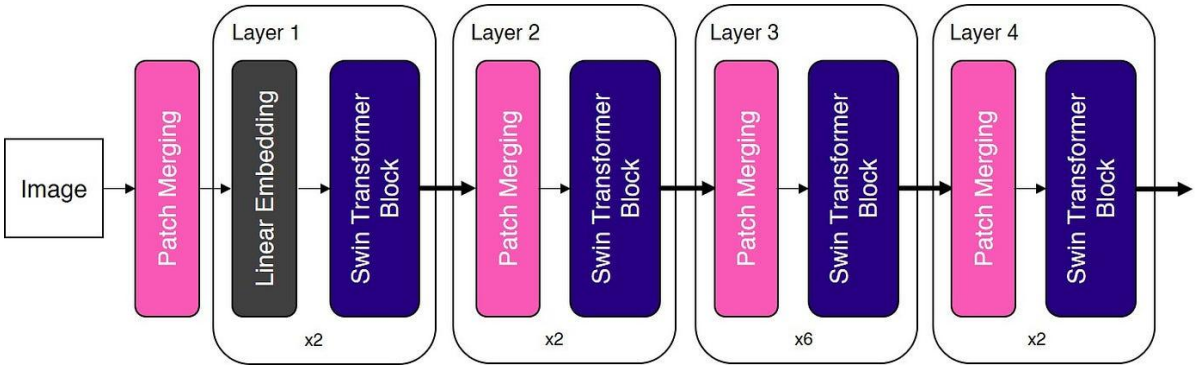


Figure 1.1. Swin Transformer

Patch Splitting: The input image is divided into fixed-size patches like putting a grid over image and each square represent a patch. Each patch is then embedded into a feature vector to form input for the transformer.

Window-Based Self-Attention: Instead of computing attention globally the model computes attention within local windows. These windows act as small focused regions capturing fine features while keeping computation manageable. Self-attention is applied within the window and captures local features.

Shifted Windows for Cross-Region Interaction: The shifted window mechanism solve limitation of local windows attention and capture global context of image. This shifted window shifts the position of the windows by a small value and hence overlapping regions with next layer. This ensure cross-window communication and improve models ability to capture global context.

Hierarchical Design: The Swin Transformer processes the image in stages:

- The image is divided into non-overlapping patches for embedding of each level.
- These patches are further split into windows and self-attention is applied locally in the window.
- The windows are shifted over next layer for overlapping and self-attention is recomputed with shifted windows.
- Hierarchical processing continues combining features to know fine details in each window without losing global context of image.

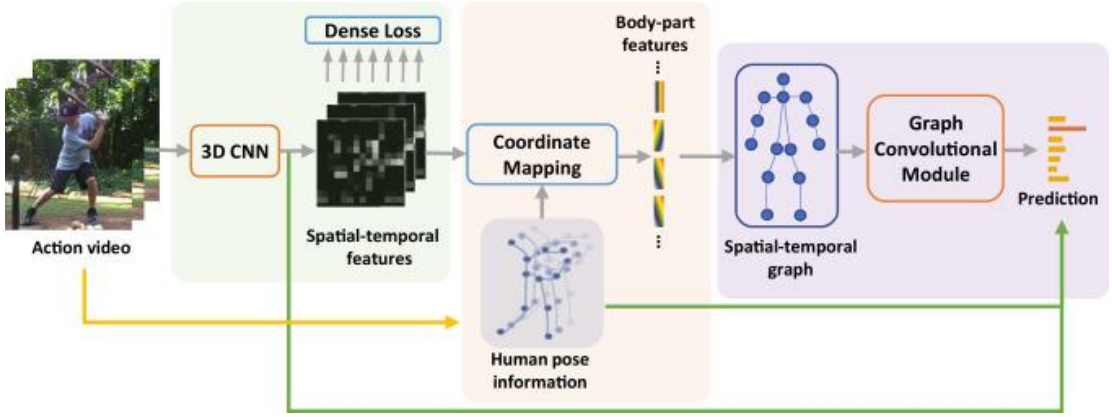
5.2.1.1.2. Applications of Swin Transformer

- **Image Classification:** Uses a hierarchical structure and multi-scale feature extraction for highly accurate image recognition.

- **Object Detection:** Detects multiple objects and fine details in an image by combining local and global context.
- **Image Segmentation:** Segments an image into different regions using robust, multi-scale feature representations.
- **Medical Imaging:** Identifies anomalies and features in high-resolution medical scans, aiding diagnostics.
- **Transfer to Natural Language Processing (NLP):** Although primarily designed for vision tasks, with architectural modifications Swin Transformer can be adapted for certain NLP applications.

5.1.2.Graph Convolutional Networks

Graph Convolutional Networks (GCNs) are a type of neural network designed to work directly with graphs. A graph consists of nodes (vertices) and edges (connections between nodes). In a GCN, each node represents an entity, and the edges represent the relationships between these entities. The primary goal of GCNs is to learn node embeddings, which are vector representations of nodes that capture the graph's structural and feature information.



1. **Input Layer**: The input layer initializes the node features, usually from raw data or pre-trained embeddings.
2. **Hidden Layers:** Hidden layers perform the graph convolution operations, progressively aggregating and transforming node features.
    - **A. Graph Convolutional Layers**: These layers perform the convolution operation on the graph. Each layer updates the feature representation of a node by aggregating the features of its neighbors.

- **B. Activation Functions**: Non-linear functions such as ReLU are applied to the output of each convolutional layer to introduce non-linearity into the model.
- **C. Pooling Layers**: These layers reduce the dimensionality of the graph by merging nodes, which helps in capturing hierarchical structures.

3. **Output Layer:** The output layer produces the final node embeddings or predictions, depending on the task (e.g., node classification, link prediction).
4. **Fully Connected Layers**: These layers are used at the end of the network to perform tasks such as classification or regression.

5.2. MYSQL 5

MySQL is a relational database management system based on the Structured Query Language, which is the popular language for accessing and managing the records in the database. MySQL is open-source and free software under the GNU license. It is supported by Oracle Company. MySQL database that provides for how to manage database and to manipulate data with the help of various SQL queries. These queries are: insert records, update records, delete records, select records, create tables, drop tables, etc. There are also given MySQL interview questions to help you better understand the MySQL database. MySQL is an open-source relational database management system (RDBMS) that uses Structured Query Language (SQL) to store, manage, and manipulate data. It is one of the most widely used database systems in web applications because of its speed, reliability, and ease of use.

MySQL is currently the most popular database management system software used for managing the relational database. It is open-source database software, which is supported by Oracle Company. It is fast, scalable, and easy to use database management system in comparison with Microsoft SQL Server and Oracle Database. It is commonly used in conjunction with PHP scripts for creating powerful and dynamic server-side or web-based enterprise applications. It is developed, marketed, and supported by MySQL AB, a Swedish company, and written in C programming language and C++ programming language. The official pronunciation of MySQL is

not the My Sequel; it is My Ess Que Ell. However, you can pronounce it in your way. Many small and big companies use MySQL. MySQL supports many Operating Systems like Windows, Linux, MacOS, etc. with C, C++, and Java languages.

Developed and maintained by Oracle Corporation.

Supports multiple platforms including Windows, Linux, and macOS.

Commonly used with programming languages like PHP, Java, and Python to build dynamic applications.

Known for its scalability, strong security features, and large developer community support.

MySQL processes client requests and interacts with stored data to execute SQL queries efficiently. It follows a structured workflow to retrieve or modify data while ensuring accuracy and reliability.

Client Request: A user sends an SQL query to the MySQL server through an application, programming interface, or command-line tool to perform a database operation.

Connection: The MySQL server establishes a connection with the client application, creating a session that allows queries to be sent and processed.

SQL Parsing: The server analyzes the SQL query to check for syntax errors and verifies whether the query structure is valid.

Query Optimization: MySQL evaluates different execution strategies and selects the most efficient way to process the query.

Execution: The server executes the query to retrieve, insert, update, or delete data from the database.

Storage Engine: MySQL uses storage engines such as InnoDB or MyISAM to manage how data is stored, retrieved, and maintained on disk.

Result Generation: After executing the query, the server prepares the requested data or confirmation message as the result.

Response: The generated result is sent back from the MySQL server to the client application.

Client Interaction: The application receives the response and displays the results to the user in a readable format.

Transaction Management: MySQL manages transactions to ensure that multiple operations are executed reliably and maintain data consistency.

Logging and Recovery: The server records logs that help recover data and restore operations if system failures occur.
Replication and Backup: MySQL supports data replication across multiple servers and regular backups to improve performance and ensure data safety.
MySQL receives a query, processes it efficiently, interacts with stored data, and returns the results while maintaining data integrity and reliability.
MySQL is a widely-used relational database management system (RDBMS) that caters to various user groups, from small businesses to large enterprises.

Small and Medium Businesses (SMBs): SMBs use MySQL to manage customer data, transactions, and business records efficiently.
Large Enterprises: Large companies use MySQL to handle large databases and high-traffic applications.
Web Developers: Developers use MySQL as a backend database for websites and web applications.
Educational Institutions: Schools and universities use MySQL for teaching database management and SQL concepts.
Applications of MySQL
MySQL is widely used across different industries due to its reliability, scalability, and performance. It supports various applications that require efficient data storage and management.

E-commerce: MySQL is used in e-commerce platforms to manage product catalogs, customer information, orders, and transactions.
Content Management Systems (CMS): Many CMS platforms use MySQL to store website content, user data, comments, and configuration settings.
Financial Services: MySQL helps manage transactional data, customer accounts, and financial records in banking and payment systems.
Healthcare: Healthcare applications use MySQL to store and manage patient records, medical histories, and treatment information.
Social Media: Many social media platforms rely on MySQL to store user profiles, posts, comments, and other interactions.

The Cloud and the Future of MySQL

Cloud technology has significantly influenced the development of MySQL, enabling better scalability, availability, and performance for modern applications.

Cloud Integration

Cloud platforms enable MySQL to run as managed services with easier deployment, scaling, and maintenance.

Managed Services: Cloud providers such as AWS, Google Cloud, and Microsoft Azure offer managed MySQL services like Amazon RDS and Google Cloud SQL to simplify database management and maintenance.

Scalability: Cloud environments allow MySQL databases to scale resources dynamically based on application demand.

Enhanced Features

Cloud-based MySQL solutions provide improved reliability, availability, and automated management features.

High Availability: Cloud-based MySQL solutions provide built-in high availability and disaster recovery options to improve reliability.

Automatic Backups: Cloud platforms offer automated backup systems that help maintain data safety and simplify recovery.

Future Trends

The future of MySQL includes deeper integration with modern cloud technologies and advanced data systems.

Hybrid Cloud Deployments: Organizations increasingly combine on-premises databases with cloud environments for better flexibility.

Advanced Analytics: MySQL is being integrated with cloud analytics and machine learning platforms to enable deeper data insights.

Serverless Architectures: MySQL may evolve to support serverless environments, making database management more efficient and cost-effective.

5.3. WAMPSERVER

WampServer is a Windows web development environment. It allows you to create web applications with Apache2, PHP and a MySQL database. Alongside, PhpMyAdmin allows you to manage easily your database.

WAMPServer is a reliable web development software program that lets you create web apps with MYSQL database and PHP Apache2. With an intuitive interface, the application features numerous functionalities and makes it the preferred choice of developers from around the world. The software is free to use and doesn't require a payment or subscription.

Windows which is an ultimate platform for beginners and advanced users to operate, process and manage the different day to day computing tasks, however, if you are a developer and want to experience some of the most powerful software without paying a single penny then you should think about Linux. There are so many software packages that are only designed to run efficiently on Linux platforms such as Apache web server, PHP interpreter and MySQL database (LAMP).

Now, the thing if we have a Windows 10/8.1/8/7 system and we don't want to change our operating system to Linux for testing Web or PHP applications or learning the curves of MySQL. In such a case, the WAMP server comes handy.

Everything listed in the abbreviation can be downloaded separately but it takes time to configure each of them. In case of WAMP, the time taken to this is much less than it comes as a whole package. It is used in web development to have a safe experience in testing features.

Configuration: Now, we have to configure the WAMP contents we installed. Once installed, you will mostly get a notification from the firewall asking whether the newly installed software should

get the permission to use your network. Give it the permission and next find the option in your hidden taskbar icons or in the windows start menu. The color of the symbol corresponds to the status the server

To set up WampServer on your Windows Server, you must go through the GUI installer wizard. To start it, double-click on the WampServer installer EXE to begin the setup process.

Once the WampServer installer EXE is open, you will be asked to choose a setup language. In the installer, select the appropriate language. Then, click the “OK” button to move to the next page in the installer.

After selecting the correct language in the installer, the WampServer installation wizard will present you with a “License Agreement”. Read it using the built-in text browser. Then, when you’ve finished reading everything, select the “I accept the agreement” button, and click “Next” to continueUpon reading the Wampserver information the installer provides, the installation tool will close. To run your Wampserver on Windows Server, simply double-click on the “Wampserver64” shortcut on the desktop, and it will launch Apache, Php, MariaDB, MySQL, etc.

5.4. BOOTSTRAP 4

Bootstrap is a free and open-source tool collection for creating responsive websites and web applications. It is the most popular HTML, CSS, and JavaScript framework for developing responsive, mobile-first websites.

It solves many problems which we had once, one of which is the cross-browser compatibility issue. Nowadays, the websites are perfect for all the browsers (IE, Firefox, and Chrome) and for all sizes of screens (Desktop, Tablets, Phablets, and Phones). All thanks to Bootstrap developers -Mark Otto and Jacob Thornton of Twitter, though it was later declared to be an open-source project.

Easy to use: Anybody with just basic knowledge of HTML and CSS can start using Bootstrap

Responsive features: Bootstrap's responsive CSS adjusts to phones, tablets, and desktops
Mobile-first approach: In Bootstrap, mobile-first styles are part of the core framework
Browser compatibility: Bootstrap 4 is compatible with all modern browsers (Chrome, Firefox, Internet Explorer 10+, Edge, Safari, and Opera)
Bootstrap is one of the popular front-end frameworks which has a nice set of predefined CSS codes. The updated version of Bootstrap5 was officially released on 16 June 2020. Bootstrap is a CSS framework that makes mobile-friendly web development and provides responsiveness.

The framework is available for free and can be utilized in two ways either by downloading the zip files and incorporating Bootstrap libraries/modules into the project or by directly including the Bootstrap URL and using the online version.
Bootstrap is the Fastest and Easiest way to create web pages.
Bootstrap is widely used to create Platform-independent web pages.
Bootstrap is popular for creating Responsive web pages.
Using pre-styled components makes development faster.
Bootstrap is based on the mobile-first concept that works well on all screen sizes.
Bootstrap makes it easy to Use the grid system for a well-organized and responsive page layout without complex coding.

Bootstrap offers pre-built components and styles, making projects faster to complete.
Large and active community support.Responsive features make it easy to create mobile-friendly websites without extra custom coding and media queries.Bootstrap offers efficient web development with its pre-designed components and responsive grid system.Bootstrap 5 Typography styles and formats text, providing customized headings, subheadings, lists, paragraphs, and font styles. It supports global settings for font stack, headings, link styles, ensuring consistency across devices.Rapid prototyping. Bootstrap is ideal for quickly building functional prototypes. Its ready-to-use components let you assemble a working interface in a short time, making it easier to test ideas and gather feedback early in the development cycle.
Responsive by default. The framework is built with a mobile-first approach, making it easy to create layouts that adapt seamlessly to various screen sizes.

Consistency. It provides a consistent design language and functionality across all modern web browsers, reducing cross-browser compatibility issues.

Strong community and documentation. Bootstrap has extensive documentation and a massive community, making it easy to find solutions, tutorials, and third-party themes.

Generic design. Without significant customization, websites built with Bootstrap can look very similar to one another, lacking a unique visual identity.

Potential bloat. The framework includes a large amount of CSS and JavaScript to cover all its components. If you only use a few features, the unused code can increase your site's file size and slow it down.

Complex overrides. While Bootstrap is customizable, overriding its default styles for highly unique designs can sometimes be complex and require deep CSS specificity.

Bootstrap is an excellent choice for projects where speed and a reliable foundation are more important than a unique brand identity. It's ideal for:Prototypes and projects with tight deadlines. Admin dashboards and internal tools.Standard marketing or corporate websites.

If you have limited front-end resources or a team with varying CSS skills, Bootstrap's pre-built components and extensive documentation provide a huge advantage.

If your project's success depends on a highly specific or unconventional design, you might find yourself fighting Bootstrap's default styles.In these cases, a utility-first framework like Tailwind CSS might offer more creative freedom.

For very simple websites, the entire framework can be overkill, adding unnecessary weight where a lighter library or custom CSS would be more efficient.Ultimately, evaluate your project's priorities. If you value speed, consistency, and a proven, responsive foundation, Bootstrap remains one of the best tools available.

5.5. FLASK

Flask is a web framework. This means flask provides you with tools, libraries and technologies that allow you to build a web application. This web application can be some web pages, a blog, a wiki or go as big as a web-based calendar application or a commercial website.

Flask is often referred to as a micro framework. It aims to keep the core of an application simple yet extensible. Flask does not have built-in abstraction layer for database handling, nor does it have formed a validation support. Instead, Flask supports the extensions to add such functionality to the application. Although Flask is rather young compared to most Python frameworks, it holds a great promise and has already gained popularity among Python web developers. Let's take a closer look into Flask, so-called "micro" framework for Python. Flask is part of the categories of the micro-framework. Micro-framework are normally framework with little to no dependencies to external libraries. This has pros and cons. Pros would be that the framework is light, there are little dependency to update and watch for security bugs, cons is that some time you will have to do more work by yourself or increase yourself the list of dependencies by adding plugins.
Flask is a lightweight WSGI web application framework. It is designed to make getting started quick and easy, with the ability to scale up to complex applications.

Get started with Installation and then get an overview with the Quickstart. There is also a more detailed Tutorial that shows how to create a small but complete application with Flask. Common patterns are described in the Patterns for Flask section. The rest of the docs describe each component of Flask in detail, with a full reference in the API section.

Flask depends on the Werkzeug WSGI toolkit, the Jinja template engine, and the Click CLI toolkit. Be sure to check their documentation as well as Flask's when looking for information.
These distributions will be installed automatically when installing Flask.

Werkzeug implements WSGI, the standard Python interface between applications and servers.

Jinja is a template language that renders the pages your application serves.

MarkupSafe comes with Jinja. It escapes untrusted input when rendering templates to avoid injection attacks.

ItsDangerous securely signs data to ensure its integrity. This is used to protect Flask's session cookie.

Click is a framework for writing command line applications. It provides the flask command and allows adding custom management commands.

Blinker provides support for Signals.
Optional dependencies-These distributions will not be installed automatically. Flask will detect and use them if you install them.
python-dotenv enables support for Environment Variables From dotenv when running flask commands.
Watchdog provides a faster, more efficient reloader for the development serverYou may choose to use Async with Gevent with your application. In this case, greenlet>=1.0 is required. When using PyPy, PyPy>=7.3.7 is required.

These are not minimum supported versions, they only indicate the first versions that added necessary features. You should use the latest versions of each.

Virtual environments-Use a virtual environment to manage the dependencies for your project, both in development and in production.

What problem does a virtual environment solve? The more Python projects you have, the more likely it is that you need to work with different versions of Python libraries, or even Python itself. Newer versions of libraries for one project can break compatibility in another project.

Virtual environments are independent groups of Python libraries, one for each project. Packages installed for one project will not affect other projects or the operating system's packages.

Python comes bundled with the venv module to create virtual environments.

Install Flask

Within the activated environment, use the following command to install Flask:

$ pip install Flask

Flask is now installed. Check out the Quickstart or go to the Documentation Overview.

A note on cookie-based sessions: Flask will take the values you put into the session object and serialize them into a cookie. If you are finding some values do not persist across requests, cookies are indeed enabled, and you are not getting a clear error message, check the size of the cookie in your page responses compared to the size supported by web browsers.

Besides the default client-side based sessions, if you want to handle sessions on the server-side instead, there are several Flask extensions that support this.

Message Flashing-Good applications and user interfaces are all about feedback. If the user does not get enough feedback they will probably end up hating the application. Flask provides a really simple way to give feedback to a user with the flashing system. The flashing system basically makes it possible to record a message at the end of a request and access it on the next (and only the next) request. This is usually combined with a layout template to expose the message.

To flash a message use the flash() method, to get hold of the messages you can use get_flashed_messages() which is also available in the templates. See Message Flashing for a full example.

Logging-Changelog,Sometimes you might be in a situation where you deal with data that should be correct, but actually is not. For example you may have some client-side code that sends an HTTP request to the server but it’s obviously malformed. This might be caused by a user tampering with the data, or the client code failing. Most of the time it’s okay to reply with 400 Bad Request in that situation, but sometimes that won’t do and the code has to continue working.

You may still want to log that something fishy happened. This is where loggers come in handy. As of Flask 0.3 a logger is preconfigured for you to use.

Accessing Request Data

For web applications it's crucial to react to the data a client sends to the server. In Flask this information is provided by the global request object. If you have some experience with Python you might be wondering how that object can be global and how Flask manages to still be threadsafe. The answer is context locals:

Context Locals:Insider Information

If you want to understand how that works and how you can implement tests with context locals, read this section, otherwise just skip it.

Certain objects in Flask are global objects, but not of the usual kind. These objects are actually proxies to objects that are local to a specific context. What a mouthful. But that is actually quite easy to understand.

Imagine the context being the handling thread. A request comes in and the web server decides to spawn a new thread (or something else, the underlying object is capable of dealing with concurrency systems other than threads). When Flask starts its internal request handling it figures out that the current thread is the active context and binds the current application and the WSGI environments to that context (thread). It does that in an intelligent way so that one application can invoke another application without breaking.

So what does this mean to you? Basically you can completely ignore that this is the case unless you are doing something like unit testing. You will notice that code which depends on a request object will suddenly break because there is no request object. The solution is creating a request object yourself and binding it to the context. The easiest solution for unit testing is to use the test_request_context() context manager. In combination with the with statement it will bind a test request so that you can interact with it.

While it’s designed to give a good starting point, the tutorial doesn’t cover all of Flask’s features. Check out the Quickstart for an overview of what Flask can do, then dive into the docs to find out more. The tutorial only uses what’s provided by Flask and Python. In another project, you might decide to use Extensions or other libraries to make some tasks simpler.

Flask is flexible. It doesn’t require you to use any particular project or code layout. However, when first starting, it’s helpful to use a more structured approach. This means that the tutorial will require a bit of boilerplate up front, but it’s done to avoid many common pitfalls that new developers encounter, and it creates a project that’s easy to expand on. Once you become more comfortable with Flask, you can step out of this structure and take full advantage of Flask’s flexibility.

SYSTEM IMPLEMENTATION

6.1. SOURCE CODE

Packages

```
from flask import Flask, render_template, Response, redirect, request, session, abort, url_for
from camera import VideoCamera
from datetime import datetime
from random import randint
import math
import cv2
import matplotlib.pyplot as plt
import pandas as pd
import numpy as np
import threading
import os
import time
import shutil
import imagehash
import PIL.Image
from PIL import Image
import argparse
import mediapipe as mp
import mysql.connector
```

Database Connection

```
mydb = mysql.connector.connect(
host="localhost",
user="root",
passwd="",
charset="utf8",
database="smart_surveillance"
```

Login

```
def login():
result=""
ff1=open("photo.txt","w")
ff1.write("1")
ff1.close()
if request.method == 'POST':
username1 = request.form['uname']
password1 = request.form['pass']
mycursor = mydb.cursor()
mycursor.execute("SELECT count(*) FROM admin where username=%s && password=%s &&
name='admin'",(username1,password1))
myresult = mycursor.fetchone()[0]
if myresult>0:
result=" Your Logged in sucessfully**"
return redirect(url_for('admin'))
else:
result="Your logged in fail!!!"
```

Add Police Contact

```
def add_contact():
vid=""
msg=""
data=[]
act = request.args.get('act')
mycursor = mydb.cursor()
if request.method=='POST':
name=request.form['name']
station=request.form['station']
mobile=request.form['mobile']
email=request.form['email']
area=request.form['area']
```

```
city=request.form['city']
#cursor.execute("update admin set name=%s, mobile=%s, email=%s, location=%s where
username='admin'", (name, mobile, email, location))
#mydb.commit()
mycursor.execute("SELECT max(id)+1 FROM ss_police")
maxid = mycursor.fetchone()[0]
if maxid is None:
maxid=1
sql = "INSERT INTO ss_police(id,name,station,mobile,email,area,city) VALUES val =
(maxid,name,station,mobile,email,area,city)
print(sql)
mycursor.execute(sql, val)
mydb.commit()
msg="success"
mycursor.execute("SELECT * FROM ss_police")
data = mycursor.fetchall()
if act=="del":
did=request.args.get("did")
mycursor.execute("delete from ss_police where id=%s",(did,))
mydb.commit()
return redirect(url_for('add_contact'))
```

Training

```
path_main = 'static/dataset'
for fname in os.listdir(path_main):
dimg.append(fname)
#list_of_elements = os.listdir(os.path.join(path_main, folder))
#resize
img = cv2.imread('static/data/'+fname)
rez = cv2.resize(img, (400, 300))
cv2.imwrite("static/dataset/"+fname, rez)
img = cv2.imread('static/dataset/'+fname)
```

```
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
cv2.imwrite("static/training/g_"+fname, gray)
##noice
img = cv2.imread('static/training/g_'+fname)
dst = cv2.fastNlMeansDenoisingColored(img, None, 10, 10, 7, 15)
fname2='ns_'+fname
cv2.imwrite("static/training/"+fname2, dst)
#binarization
image = cv2.imread('static/dataset/'+fname)
original = image.copy()
kmeans = kmeans_color_quantization(image, clusters=4)
# Convert to grayscale, Gaussian blur, adaptive threshold
gray = cv2.cvtColor(kmeans, cv2.COLOR_BGR2GRAY)
blur = cv2.GaussianBlur(gray, (3,3), 0)
thresh = cv2.adaptiveThreshold(blur,255,cv2.ADAPTIVE_THRESH_GAUSSIAN_C,
cv2.THRESH_BINARY_INV,21,2)
# Draw largest enclosing circle onto a mask
mask = np.zeros(original.shape[:2], dtype=np.uint8)
cnts = cv2.findContours(thresh, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
cnts = cnts[0] if len(cnts) == 2 else cnts[1]
cnts = sorted(cnts, key=cv2.contourArea, reverse=True)
for c in cnts:
((x, y), r) = cv2.minEnclosingCircle(c)
cv2.circle(image, (int(x), int(y)), int(r), (36, 255, 12), 2)
cv2.circle(mask, (int(x), int(y)), int(r), 255, -1)
break
# Bitwise-and for result
result = cv2.bitwise_and(original, original, mask=mask)
result[mask==0] = (0,0,0)
cv2.imshow('thresh', thresh)
cv2.imshow('result', result)
```

```
#RPN
img = cv2.imread('static/training/g_'+fname)
#img = cv2.imread('static/trained/seg/'+fn2)
gray = cv2.cvtColor(img,cv2.COLOR_BGR2GRAY)
ret, thresh = cv2.threshold(gray,0,255,cv2.THRESH_BINARY_INV+cv2.THRESH_OTSU)
kernel = np.ones((3,3),np.uint8)
opening = cv2.morphologyEx(thresh,cv2.MORPH_OPEN,kernel, iterations = 2)
# sure background area
sure_bg = cv2.dilate(opening,kernel,iterations=3)
# Finding sure foreground area
dist_transform = cv2.distanceTransform(opening,cv2.DIST_L2,5)
ret, sure_fg = cv2.threshold(dist_transform,1.5*dist_transform.max(),255,0)
# Finding unknown region
sure_fg = np.uint8(sure_fg)
segment = cv2.subtract(sure_bg,sure_fg)
img = Image.fromarray(img)
segment = Image.fromarray(segment)
path3="static/training/sg/"+fname
segment.save(path3)
def get_frame(self):
success, image = self.video.read()
#self.out.write(image)
self.k+=1
cv2.imwrite("static/getimg.jpg", image)
gg="f"+str(self.k)+".jpg"
h=1
gray_image = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
#open the template as gray scale image
template = cv2.imread("static/t1/t"+str(h)+".jpg", 0)
width, height = template.shape[::-1]
match = cv2.matchTemplate(gray_image, template, cv2.TM_CCOEFF_NORMED)
```

```
threshold = 0.8
position = np.where(match >= threshold)
j=0
for point in zip(*position[::-1]):
cv2.putText(image, "Fight", (point[0]+5,point[1]-20), cv2.FONT_HERSHEY_SIMPLEX, 1,
(0,0,255), 1)
cv2.rectangle(image, point, (point[0] + width, point[1] + height), (0, 0, 255), 0)
#THREAT LEVEL MAPPING
THREAT_OBJECTS = {
"weapon": "HIGH",
"knife": "HIGH",
"gun": "HIGH",
"intruder": "MEDIUM",
"suspicious_package": "MEDIUM",
"vehicle": "LOW",
"person": "LOW"
ABNORMAL_ACTIVITIES = {
"fighting": "HIGH",
"loitering": "MEDIUM",
"running_abnormal": "MEDIUM",
"falling": "HIGH",
"normal": "LOW"
#OBJECT + THREAT DETECTION (Swin-T Placeholder)
class SwinThreatDetector:
def __init__(self, device="cpu"):
self.device = device
# Example:
# self.model = timm.create_model("swin_tiny_patch4_window7_224", pretrained=True)
# self.model.eval().to(self.device)
def detect(self, frame):
Return list of detections:
```

```
{"label": "person", "confidence": 0.9, "bbox": (x1,y1,x2,y2)}
h, w, _ = frame.shape

# ---- Dummy Example Detection (Replace with actual Swin-T detector output)
detections = [
{"label": "person", "confidence": 0.88, "bbox": (int(w*0.2), int(h*0.2), int(w*0.4), int(h*0.8))}
return detections
#POSE EXTRACTION + ACTIVITY RECOGNITION (Pose-GCN Placeholder)
class PoseGCNActivityRecognizer:
def __init__(self, device="cpu"):
self.device = device
self.pose = mp.solutions.pose.Pose(static_image_mode=False,
model_complexity=1,
enable_segmentation=False,
min_detection_confidence=0.5,
min_tracking_confidence=0.5)
# Pose sequence buffer
self.sequence = []
self.max_seq_len = 30
def extract_keypoints(self, frame):
rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
results = self.pose.process(rgb)
if not results.pose_landmarks:
return None
keypoints = []
for lm in results.pose_landmarks.landmark:
keypoints.append([lm.x, lm.y, lm.z, lm.visibility])
return np.array(keypoints, dtype=np.float32)
def classify_activity(self, keypoints):
if keypoints is None:
return "normal", 0.0
```

```
self.sequence.append(keypoints)
if len(self.sequence) > self.max_seq_len:
self.sequence.pop(0)
if len(self.sequence) >= 10:
motion = np.var(np.array(self.sequence)[-10:, :, :2])
if motion > 0.002:
return "running_abnormal", min(1.0, motion * 200)
return "normal", 0.2
#RISK SCORING + SEVERITY
def compute_threat_score(detections):
max_score = 0.0
threat_label = "none"
for det in detections:
label = det["label"]
conf = det["confidence"]
if label in ["weapon", "knife", "gun"]:
score = min(1.0, conf + 0.2)
elif label in ["intruder", "suspicious_package"]:
score = conf
else:
score = conf * 0.4
if score > max_score:
max_score = score
threat_label = label
return max_score, threat_label
def severity_from_score(score):
if score >= 0.85:
return "HIGH"
elif score >= 0.60:
return "MEDIUM"
return "LOW"
```

```
def threat_detect():
device = "cuda" if torch.cuda.is_available() else "cpu"
print("[INFO] Using device:", device)
detector = SwinThreatDetector(device=device)
activity_model = PoseGCNActivityRecognizer(device=device)
cap = cv2.VideoCapture(VIDEO_SOURCE)
if not cap.isOpened():
print("[ERROR] Cannot open video source.")
return
frame_count = 0
while True:
ret, frame = cap.read()
if not ret:
break
frame_count += 1
if frame_count % FRAME_SKIP != 0:
continue
timestamp = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
#Object detection
detections = detector.detect(frame)
threat_score, threat_label = compute_threat_score(detections)
threat_severity = severity_from_score(threat_score)
# Draw detections
for det in detections:
x1, y1, x2, y2 = det["bbox"]
label = det["label"]
conf = det["confidence"]
cv2.rectangle(frame, (x1, y1), (x2, y2), (0, 255, 0), 2)
cv2.putText(frame, f"{label} {conf:.2f}",
(x1, y1 - 10),
cv2.FONT_HERSHEY_SIMPLEX, 0.6, (0, 255, 0), 2)
```

```
#Pose behavior analysis
keypoints = activity_model.extract_keypoints(frame)
activity_label, behavior_score = activity_model.classify_activity(keypoints)
behavior_severity = severity_from_score(behavior_score)
cv2.putText(frame, f"Threat: {threat_label} ({threat_severity})",
(20, 30), cv2.FONT_HERSHEY_SIMPLEX, 0.8, (50, 50, 255), 2)
cv2.putText(frame, f"Activity: {activity_label} ({behavior_severity})",
(20, 60), cv2.FONT_HERSHEY_SIMPLEX, 0.8, (255, 50, 50), 2)
#Alerting decision
alert_triggered = False
alert_message = ""
if threat_score >= ALERT_THRESHOLD_OBJECT:
alert_triggered = True
alert_message += f"[OBJECT ALERT] {threat_label} detected | Score={threat_score:.2f}\n"
if behavior_score >= ALERT_THRESHOLD_BEHAVIOR:
alert_triggered = True
alert_message += f"[BEHAVIOR ALERT] {activity_label} detected |
Score={behavior_score:.2f}\n"
# Save snapshot + log if alert
if alert_triggered:
snapshot_name = f"alert_{datetime.now().strftime('%Y%m%d_%H%M%S')}.jpg"
snapshot_path = os.path.join(SNAPSHOT_DIR, snapshot_name)
cv2.imwrite(snapshot_path, frame)
send_sms_alert(alert_message)
send_email_alert("PatrolBot Alert", alert_message)
event_data = {
"timestamp": timestamp,
"threat_label": threat_label,
"threat_score": threat_score,
"threat_severity": threat_severity,
"activity_label": activity_label,
```

```
"behavior_score": behavior_score,
"behavior_severity": behavior_severity,
"snapshot_path": snapshot_path
log_event(event_data)
# Display output
cv2.imshow("PatrolBot - AI Surveillance", frame)
# Exit
if cv2.waitKey(1) & 0xFF == ord("q"):
break
cap.release()
cv2.destroyAllWindows()
```

SYSTEM TESTING

7.1. SOFTWARE TESTING

Software testing is a critical phase in the development of this AI-powered surveillance system. It ensures that the application operates accurately, securely, and efficiently while meeting both functional and non-functional requirements. Testing helps identify system defects, validate AI model performance, verify real-time processing capabilities, and ensure reliable alert generation. Through systematic testing, the system becomes more stable, accurate, and effective for real-time security monitoring.

7.1.1 Types of Testing

- **Unit Testing:** Unit testing is performed on individual modules to ensure each component functions correctly. Modules such as video preprocessing, object detection, behavior recognition, threat classification, alert generation, and event logging are tested independently to detect and resolve issues before system integration.
- **Integration Testing:** Integration testing verifies smooth interaction between system components. It ensures proper communication between the CCTV input module, AI detection models, decision tree logic, dashboard interface, database, and notification services, confirming accurate data flow across the system.
- **System Testing:** System testing evaluates the complete surveillance system in a real-time environment. It validates end-to-end functionalities such as live video analysis, threat detection, tracking, alert triggering, and event logging to ensure compliance with system requirements.
- **Alpha testing:** Testing is conducted from the perspective of security personnel and system administrators. Users test live monitoring, alert notifications, dashboard usability, and incident review features to ensure the system is practical, reliable, and easy to use before deployment.
- **White box Testing:** testing assesses the system's responsiveness, frame processing speed, and stability under different workloads. The system is tested with multiple camera feeds and continuous operation to ensure minimal latency and consistent real-time performance.

- **Security Testing:** Security testing ensures that surveillance data, alerts, and user access controls are protected from unauthorized access. It validates authentication mechanisms, secure data storage, and resistance to potential cyber threats.
- **Usability Testing:** Usability testing focuses on the dashboard design and user experience. It ensures that security operators can easily navigate live feeds, interpret alerts, access logs, and manage system settings without complexity.
- **Black box Testing:** testing is carried out after system updates or model retraining to ensure that new changes do not affect existing functionalities. This helps maintain system stability, accuracy, and consistency throughout development and deployment.

7.2. TEST CASES

Test Case ID: TC001

- **Input:** Live CCTV camera feed is connected and activated.
- **Expected Result:** Video stream starts successfully and monitoring begins.
- **Actual Result:** Live feed displayed and system monitoring activated.
- **Status:** Pass

Test Case ID: TC002

- **Input:** CCTV camera is configured with incorrect IP or credentials.
- **Expected Result:** System rejects configuration and displays error message.
- **Actual Result:** Invalid camera configuration message shown.
- **Status:** Pass

Test Case ID: TC003

- **Input:** CCTV footage contains a visible weapon or suspicious object.
- **Expected Result:** Object detected and classified as a potential threat.
- **Actual Result:** Threat detected and highlighted correctly.
- **Status:** Pass

Test Case ID: TC004

- **Input:** Video stream shows abnormal human behavior (fighting/loitering).
- **Expected Result:** Behavior recognized and marked as abnormal.
- **Actual Result:** Abnormal behavior detected successfully.
- **Status:** Pass

Test Case ID: TC005

- **Input:** Normal human activity (walking, standing, sitting).
- **Expected Result:** No alert generated; activity marked as safe.
- **Actual Result:** Activity classified as normal with no alerts.
- **Status:** Pass

Test Case ID: TC006

- **Input:** Confirmed critical threat detected in live video.
- **Expected Result:** SMS and email alerts sent to authorized personnel.
- **Actual Result:** Alerts delivered successfully.
- **Status:** Pass

Test Case ID: TC007

- **Input:** Multiple moving objects in crowded environment.
- **Expected Result:** Objects tracked continuously without identity loss.
- **Actual Result:** Objects tracked correctly using unique IDs.
- **Status:** Pass

Test Case ID: TC008

- **Input:** Detected threat event stored in system logs.
- **Expected Result:** Incident logged with timestamp, camera ID, and evidence.
- **Actual Result:** Event recorded successfully in database.
- **Status:** Pass

Test Case ID: TC009

- **Input:** Unauthorized user attempts to access dashboard or logs.
- **Expected Result:** Access denied and security alert triggered.
- **Actual Result:** Unauthorized access blocked successfully.
- **Status:** Pass

Test Case ID: TC010

- **Input:** Admin requests historical incident reports and analytics.
- **Expected Result:** Complete logs and statistics displayed accurately.
- **Actual Result:** Incident reports generated correctly.
- **Status:** Pass

Test Case ID: TC011

- **Input:** System processes multiple CCTV feeds simultaneously.
- **Expected Result:** Real-time detection maintained without delay.
- **Actual Result:** System handled multiple feeds efficiently.
- **Status:** Pass

Test Case ID: TC012

- **Input:** AI model updated and redeployed into the dashboard.
- **Expected Result:** Updated model integrated without affecting operations.
- **Actual Result:** Model deployed and system functioned normally.
- **Status:** Pass

7.3. TEST REPORT

Introduction

The AI-based surveillance system was thoroughly tested to ensure reliable, efficient, and accurate real-time threat detection and situational awareness in public and private environments. The testing process focused on validating live video monitoring, object detection, abnormal behavior recognition, real-time alerts, event logging, and dashboard functionality. This report summarizes the testing objectives, scope, environment, results, and identified issues to ensure that the system meets all functional, performance, and security requirements.

Test Objective

- To verify correct configuration and activation of CCTV camera feeds.
- To ensure accurate real-time object detection using Swin Transformer.
- To validate abnormal behavior recognition using Pose-GCN.
- To test instant alert notifications (SMS and email) for detected threats.
- To validate proper tracking and classification of multiple objects in crowded scenes.
- To evaluate event logging, dashboard monitoring, and audit trail accuracy.
- To assess system performance under multiple simultaneous camera feeds.
- To confirm system reliability, security, and scalability.

Test Scope

- CCTV camera setup and live feed monitoring.
- Real-time object detection and classification.
- Abnormal human behavior recognition.
- Real-time alert and notification system.
- Event logging and dashboard visualization.
- Threat assessment and decision-making using Decision Tree.
- System performance under concurrent multiple video streams.
- Security of system access, data storage, and logs.

Test Environment

- **Hardware:** Intel Core i5/i7, 8–16 GB RAM, 256–500 GB SSD, NVIDIA GPU (recommended).

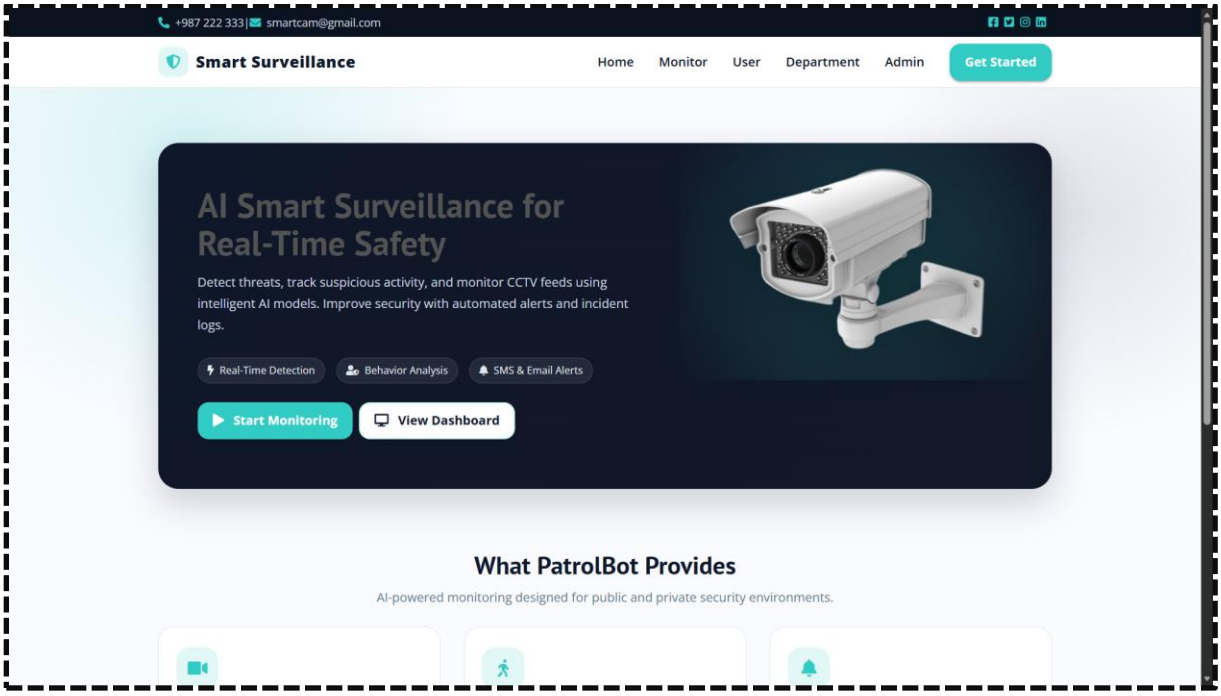
- **Software:** Windows 10/11, Python 3.8+, Flask, MySQL, Wamp Server.
- **Libraries & Tools:** OpenCV, TensorFlow, Keras, NumPy, Pandas, Scikit-learn, Bootstrap.
- **Notification Services:** SMS API, SMTP email service.
- **Browser:** Google Chrome, Microsoft Edge for dashboard access.
- **Video Datasets:** Labeled CCTV datasets for objects and human behaviors.

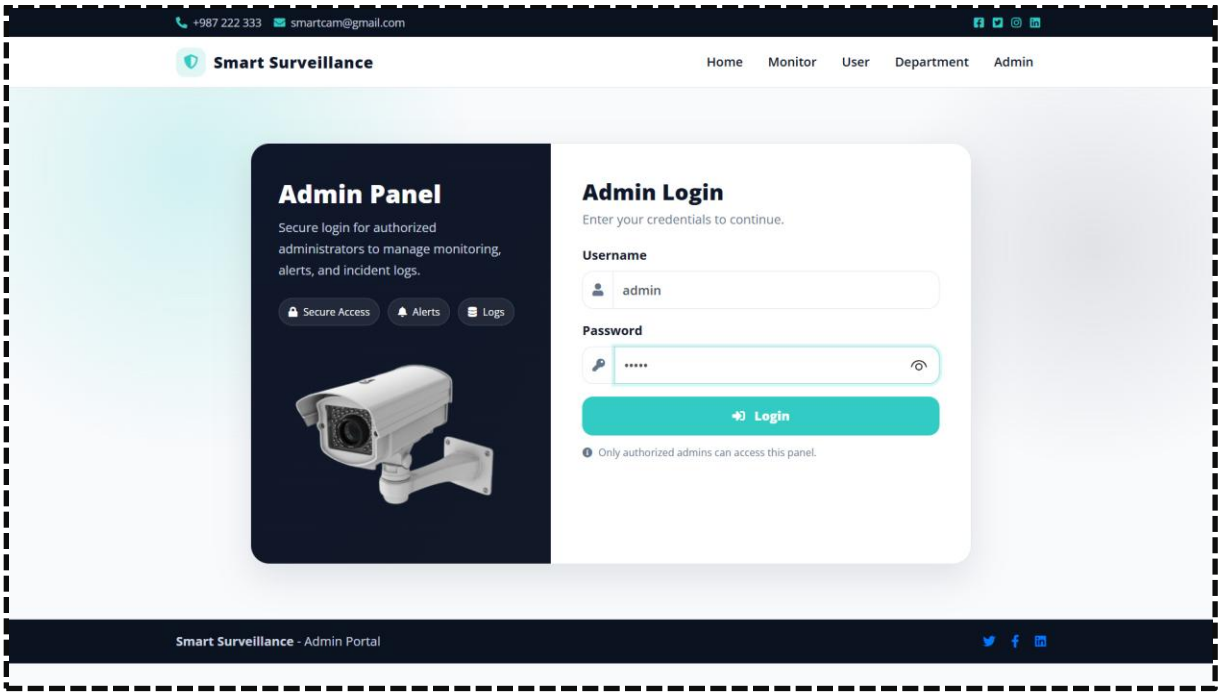
Test Conclusion

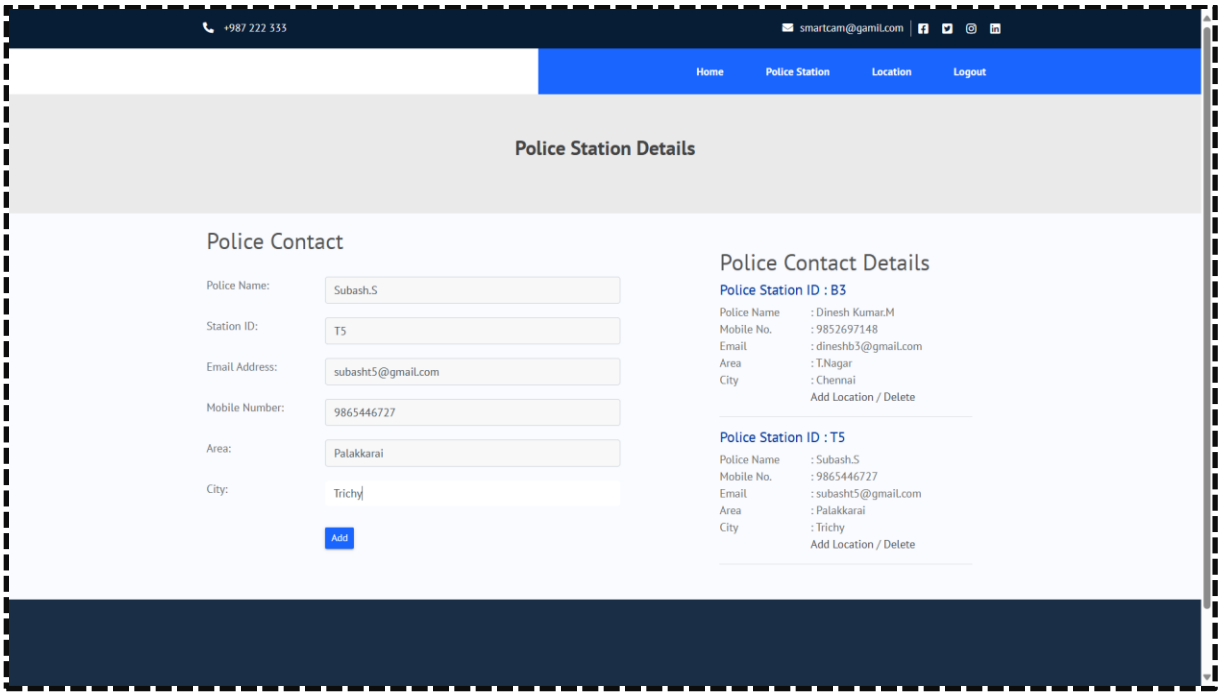
The project successfully met all functional, performance, and security requirements. Testing confirmed that live video monitoring, real-time object detection, behavior recognition, alert notifications, and event logging operate accurately and efficiently. The system maintained real-time processing even under multiple camera streams, and minor issues identified during testing were resolved to enhance stability and responsiveness. Overall, the surveillance system is ready for deployment, offering a robust, intelligent, and scalable solution for modern security management, ensuring faster threat detection, improved situational awareness, and effective emergency response.

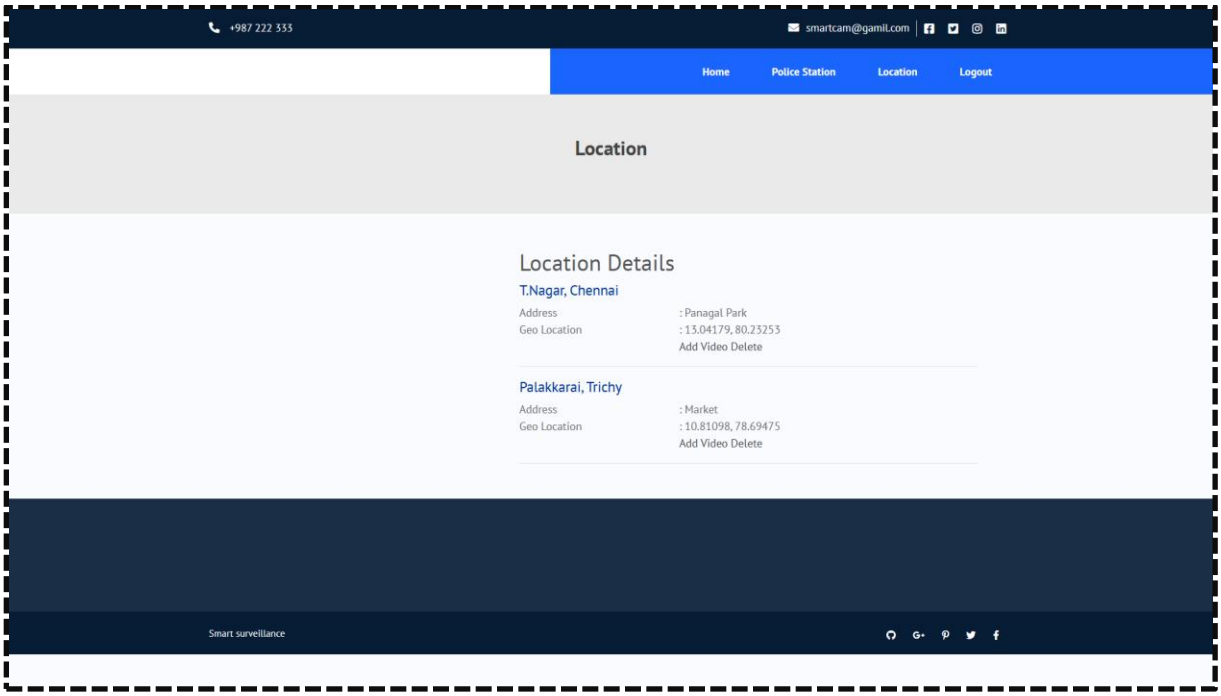
APPENDICES

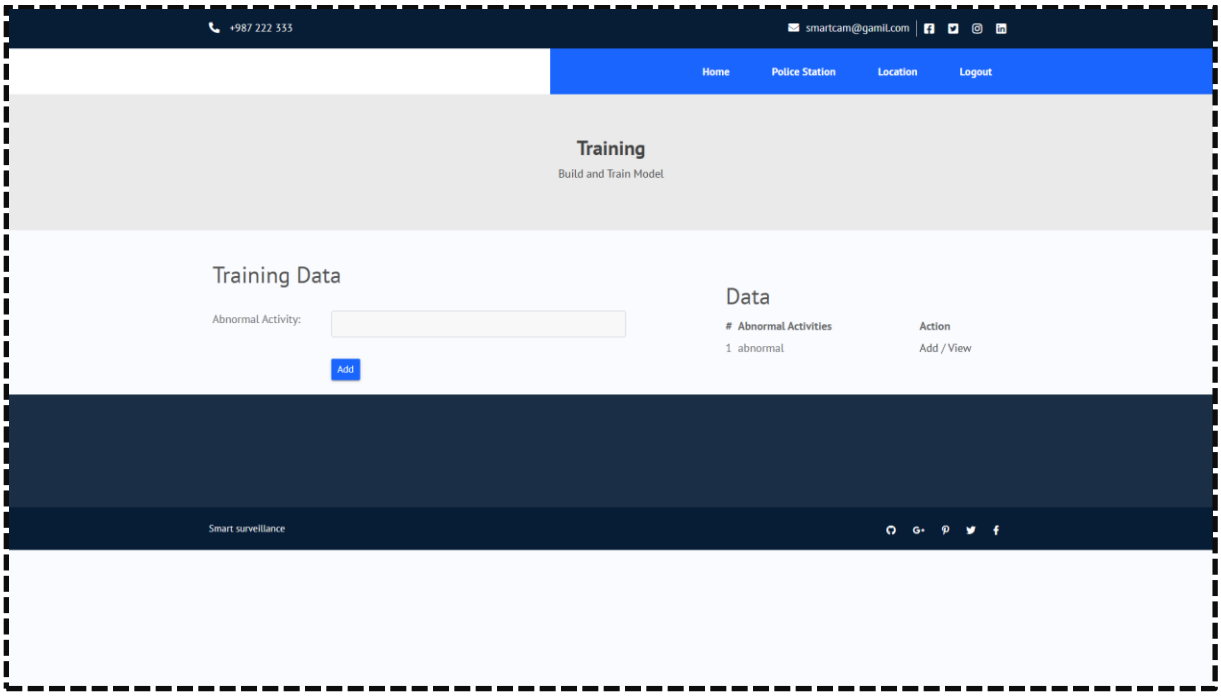
8.1. SCREENSHOTS

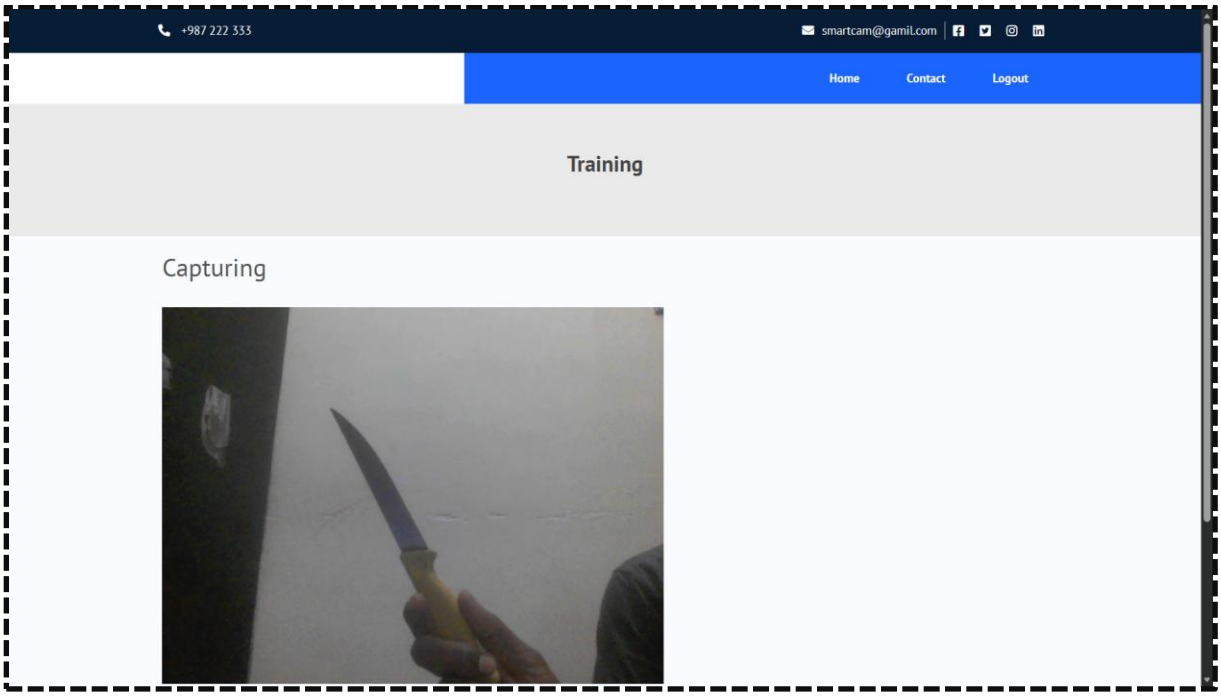


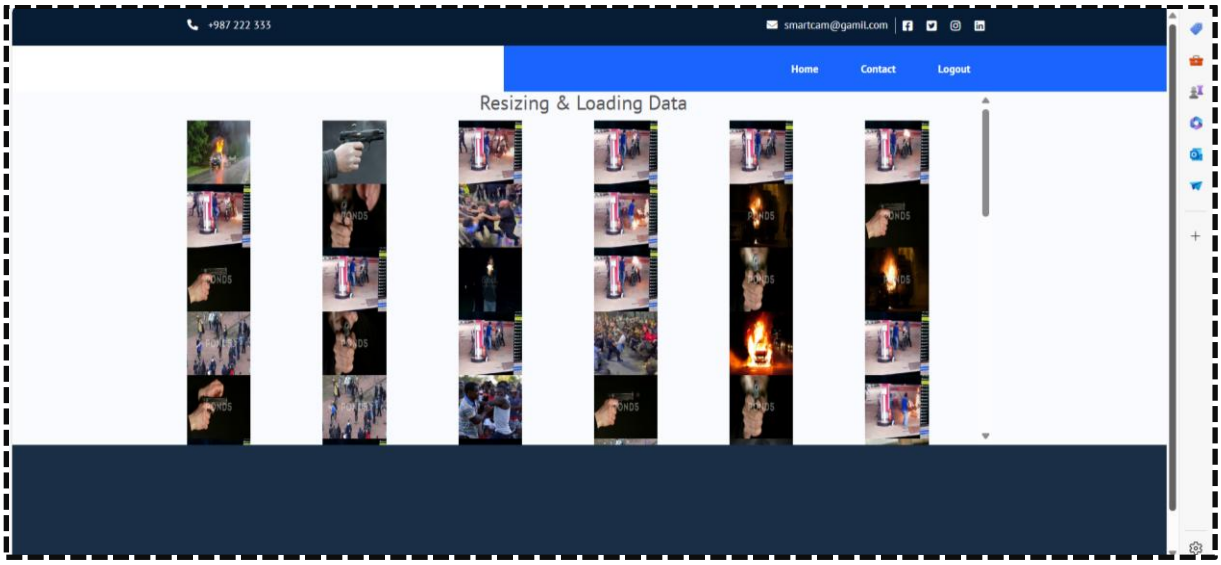


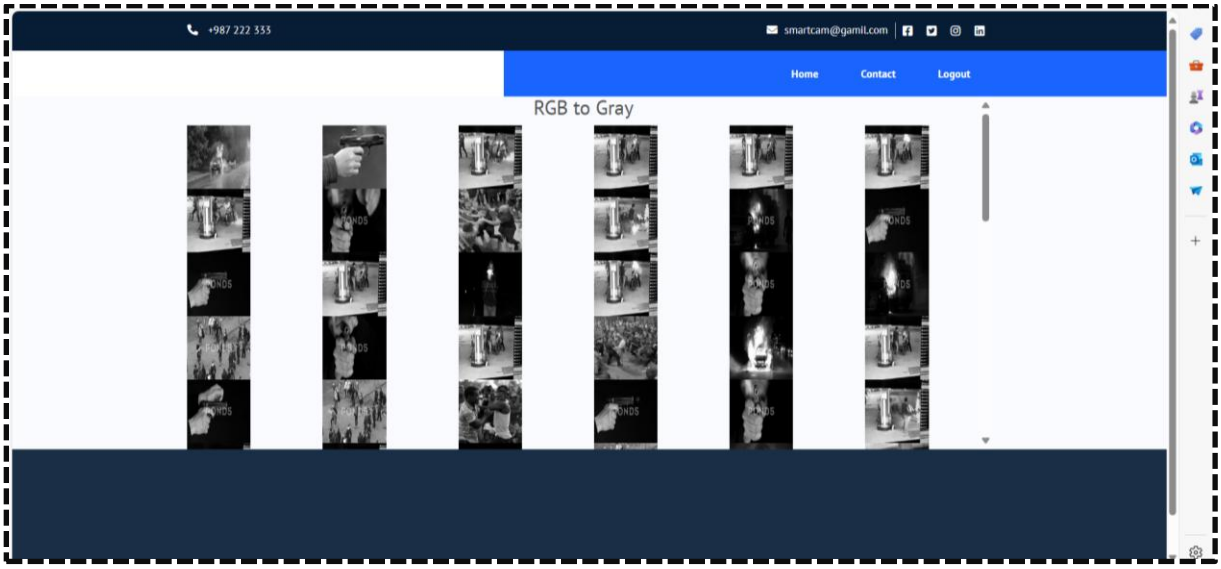

+987 222 333
smartcam@gamil.com
Home
Police Station
Location
Logout
Police Station Details
Police Contact
Police Name:
Subash.S
Station ID:
T5
Email Address:
subasht5@gmail.com
Mobile Number:
9865446727
Area:
Palakkarai
City:
Trichy
Add
Police Contact Details
Police Station ID : B3
Police Name : Dinesh Kumar.M
Mobile No. : 9852697148
Email : dineshb3@gmail.com
Area : T.Nagar
City : Chennai
Add Location / Delete
Police Station ID : T5
Police Name : Subash.S
Mobile No. : 9865446727
Email : subasht5@gmail.com
Area : Palakkarai
City : Trichy
Add Location / Delete

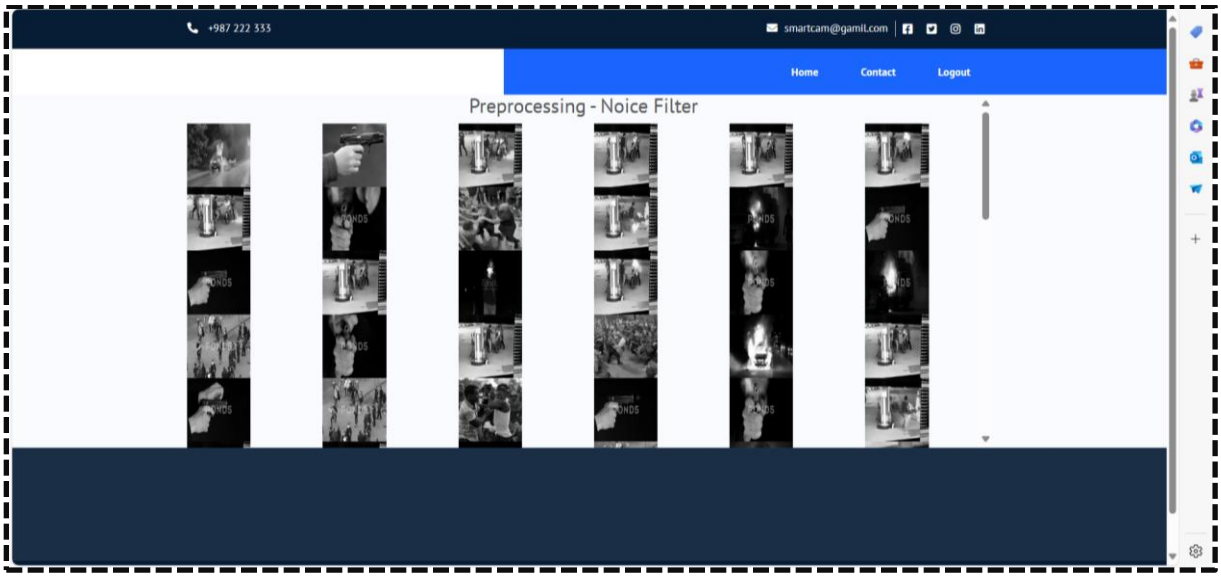

+987 222 333
smartcam@gamil.com
Home
Police Station
Location
Logout
Location
Location Details
T.Nagar, Chennai
Address : Panagal Park
Geo Location : 13.04179, 80.23253
Add Video Delete
Palakkarai, Trichy
Address : Market
Geo Location : 10.81098, 78.69475
Add Video Delete
Smart surveillance

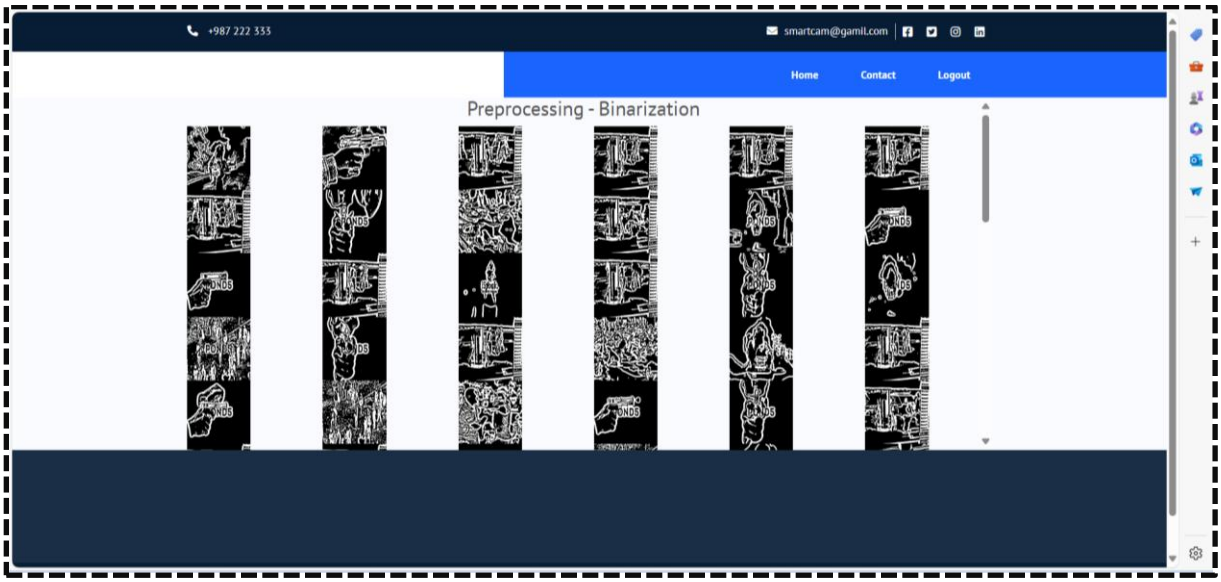

+987 222 333
smartcam@gamil.com
Home
Police Station
Location
Logout
Training
Build and Train Model
Training Data
Abnormal Activity:
Add
Data
Abnormal Activities
Action
1 abnormal
Add / View
Smart surveillance


+987 222 333
smartcam@gamil.com
Home
Contact
Logout
Training
Capturing


+987 222 333
smartcam@gamil.com
Home
Contact
Logout
Resizing & Loading Data

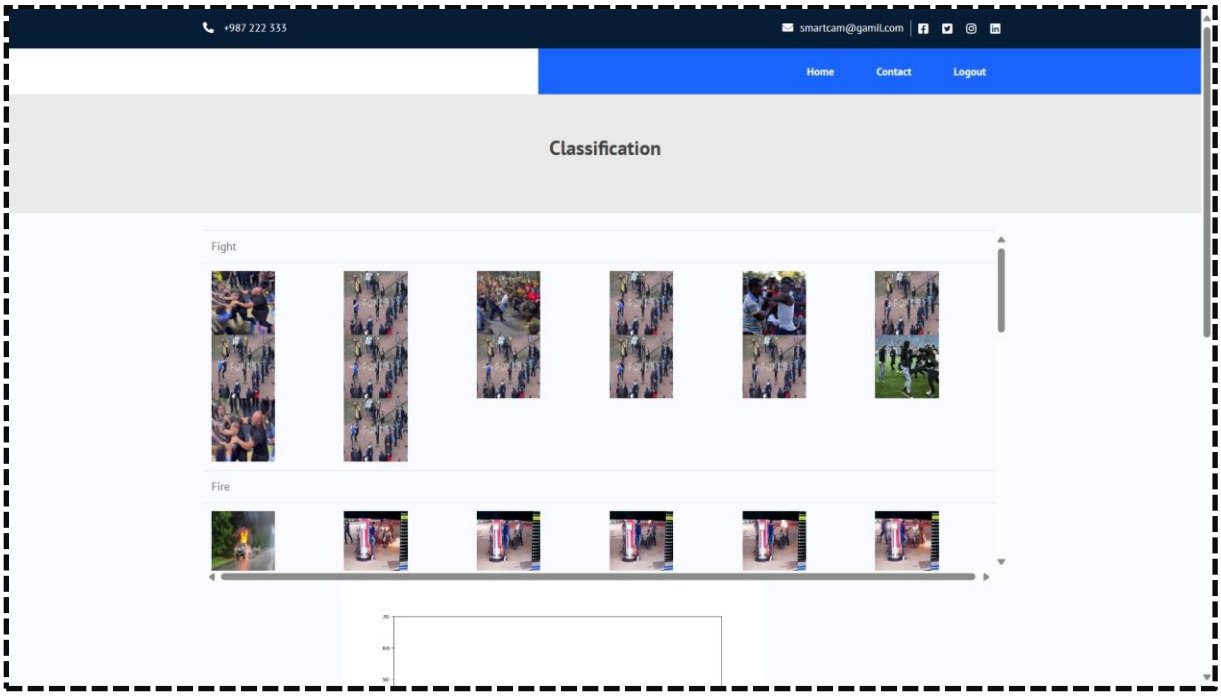

+987 222 333
smartcam@gamil.com
Home
Contact
Logout
RGB to Gray

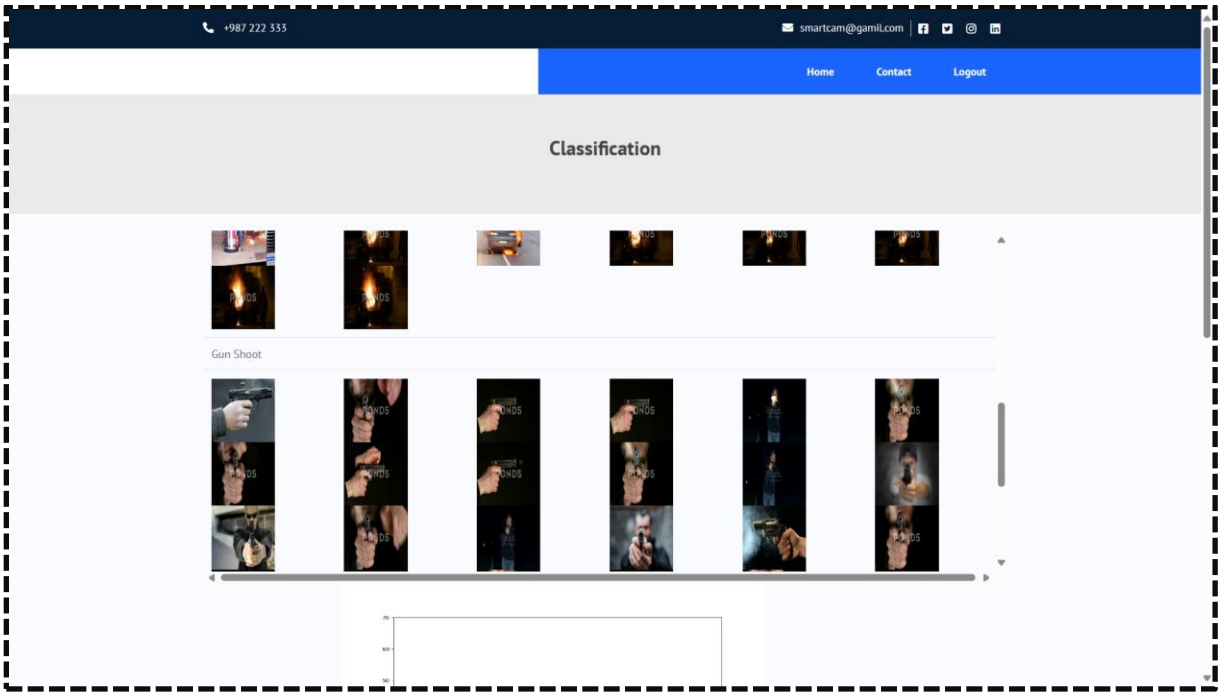

+987 222 333
smartcam@gamil.com
Home
Contact
Logout
Preprocessing - Noice Filter

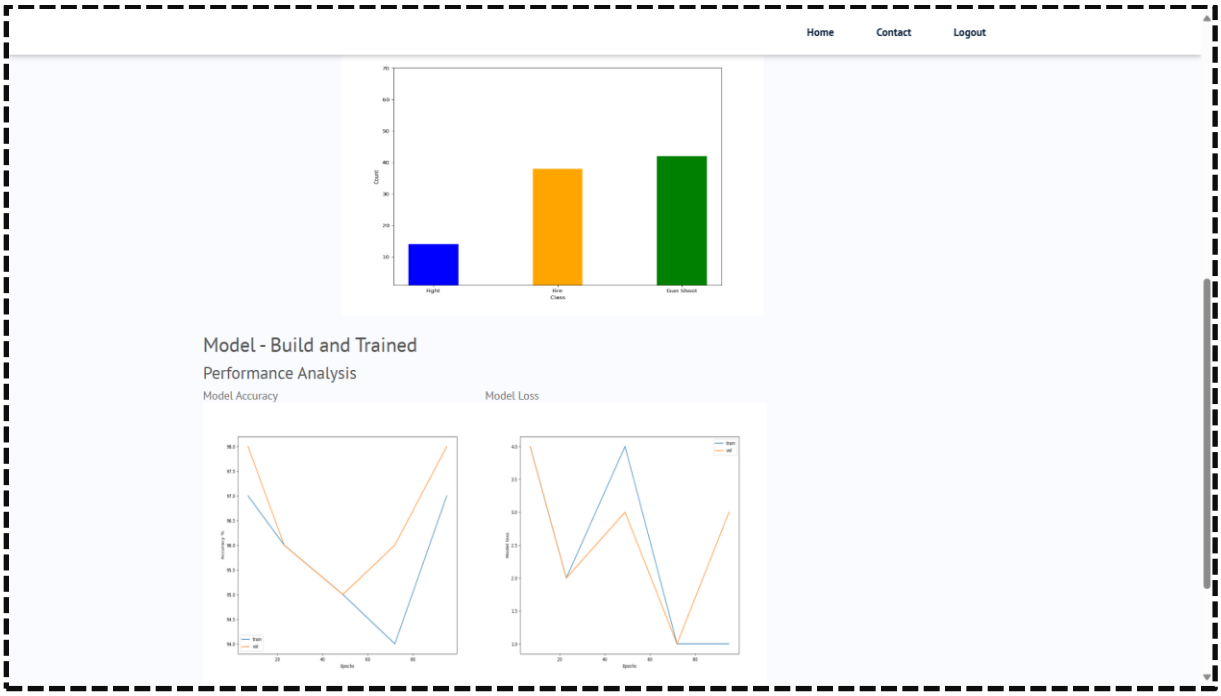

+987 222 333
smartcam@gamil.com
Home
Contact
Logout
Preprocessing - Binarization

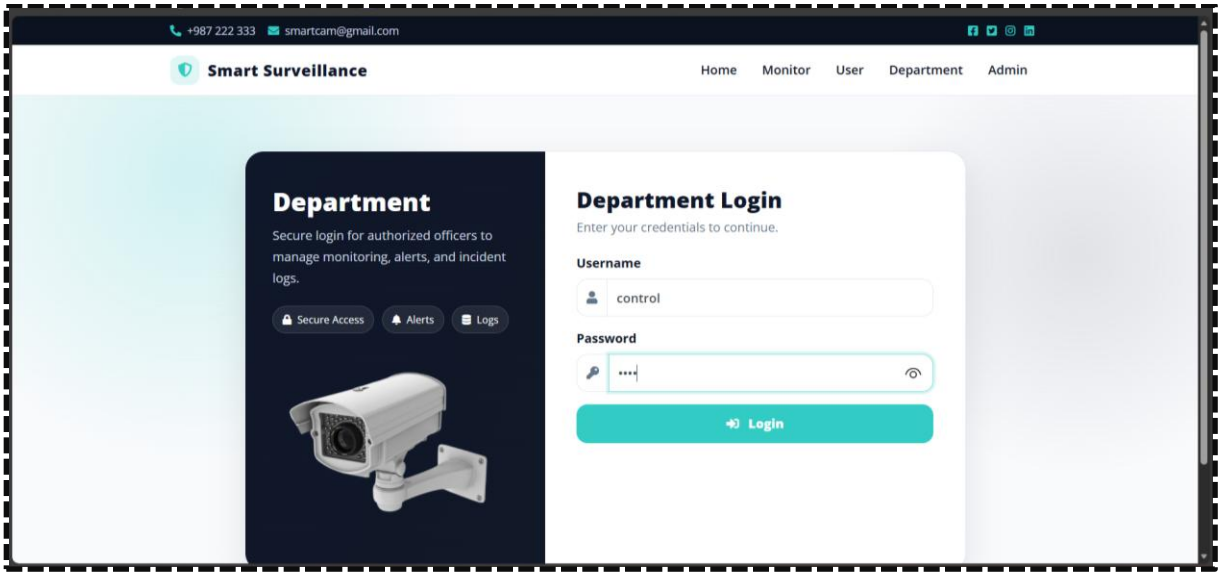

+987 222 333
smartcam@gamil.com
Home
Contact
Logout
Segmentation - RPN

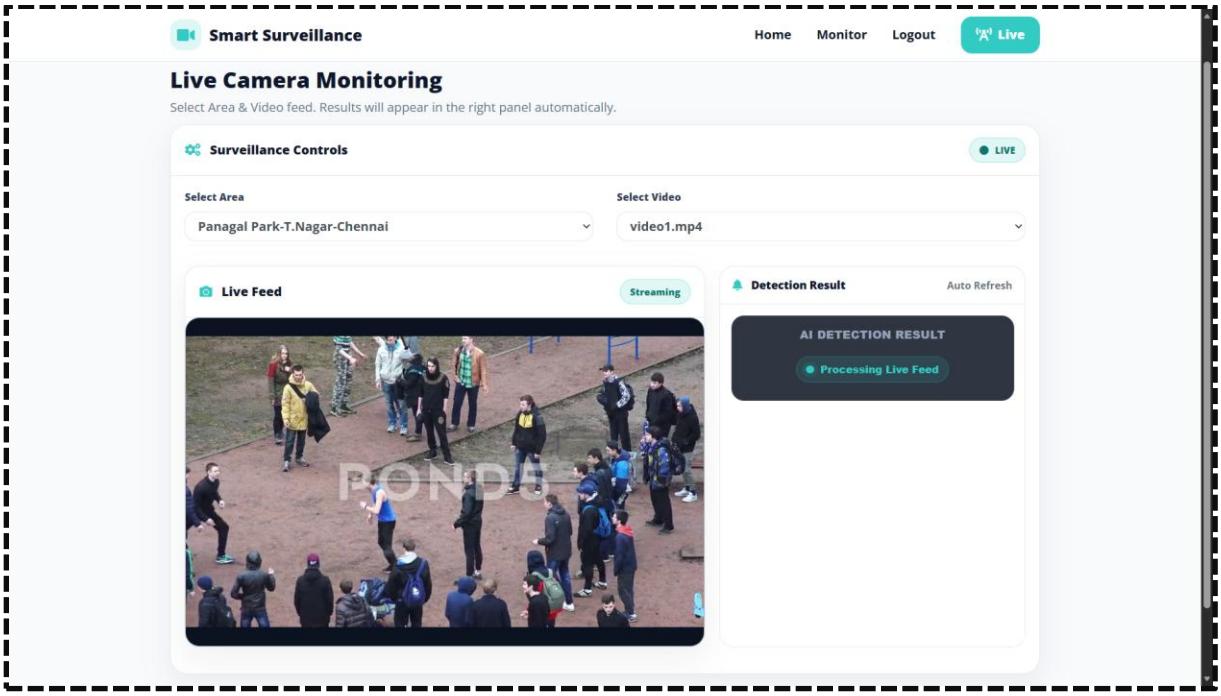

+987 222 333
smartcam@gamil.com
Home
Contact
Logout
Feature Extraction

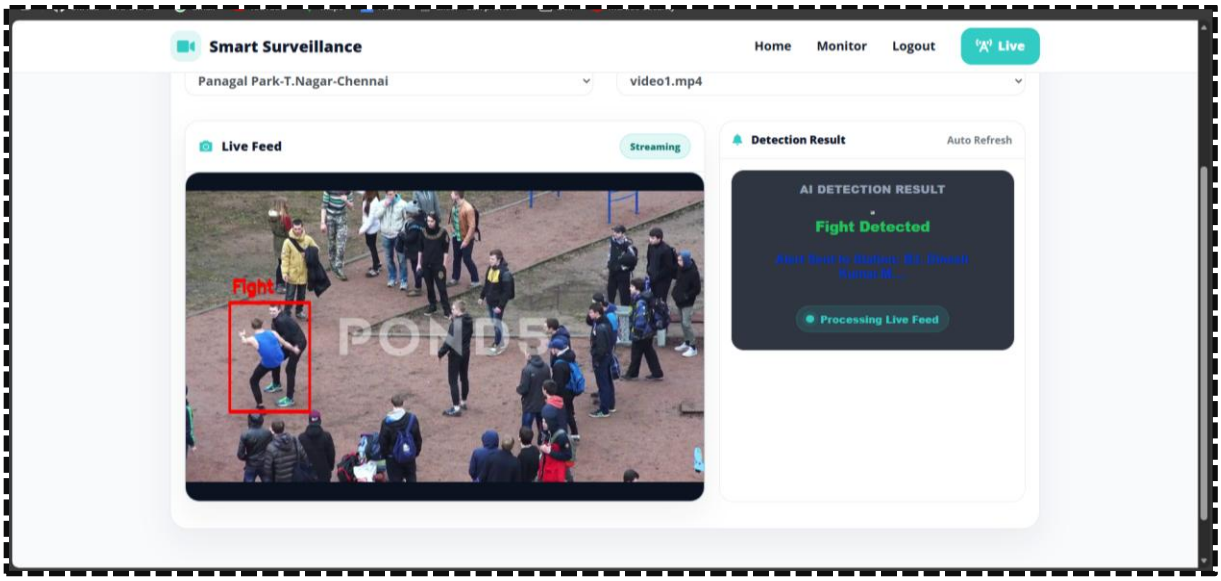

+987 222 333
smartcam@gamil.com
Home
Contact
Logout
Classification
Fight
Fire

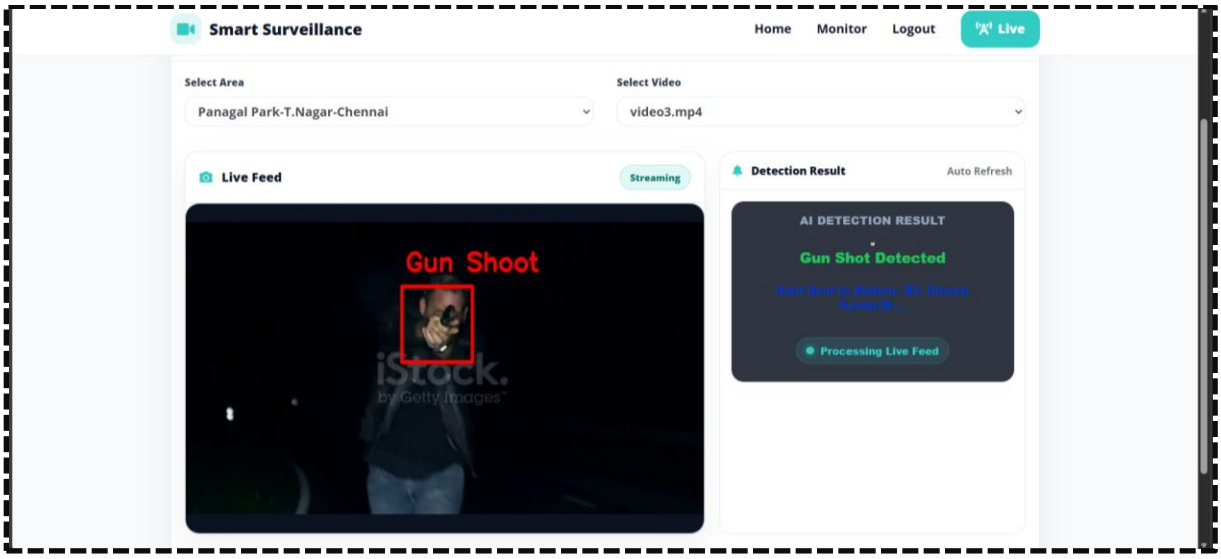

+987 222 333
smartcam@gamil.com
Home
Contact
Logout
Classification
Gun Shoot

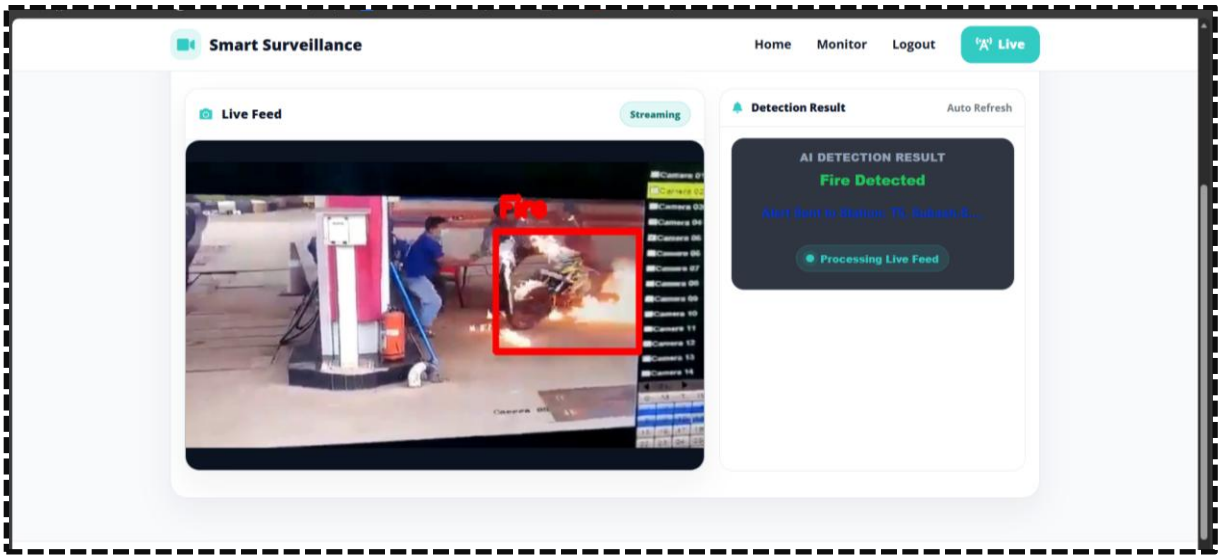

Home
Contact
Logout
Model - Build and Trained
Performance Analysis
Model Accuracy
Model Loss

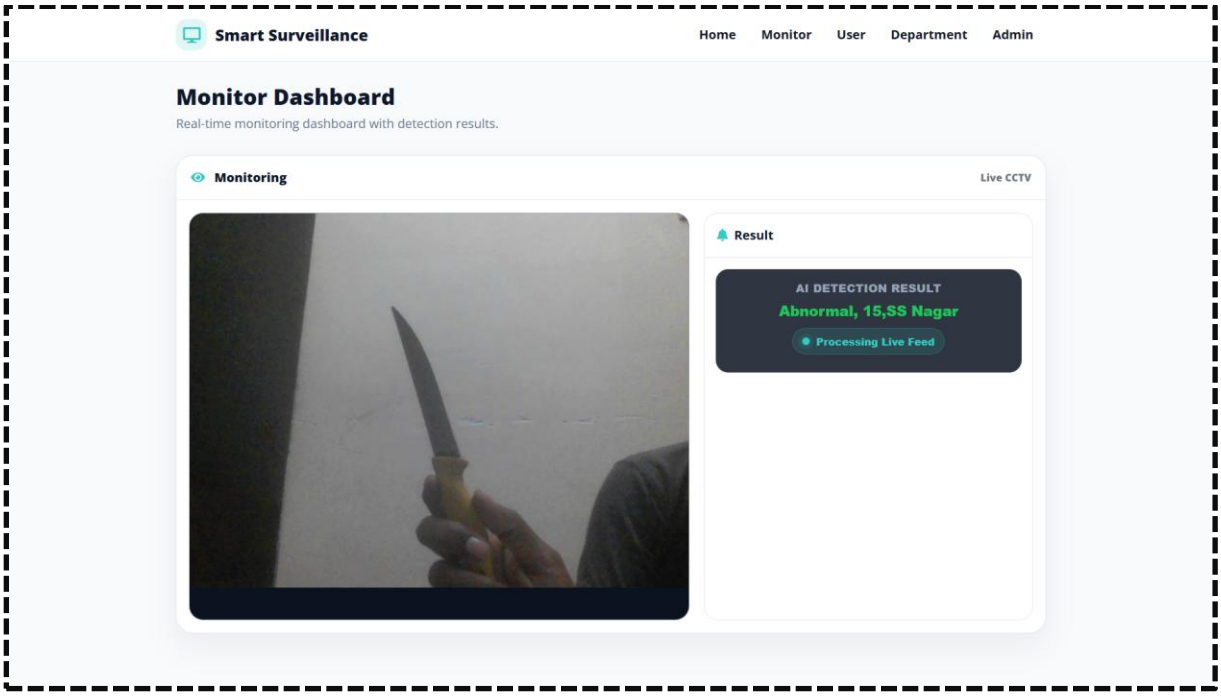

+987 222 333
smartcam@gmail.com
Smart Surveillance
Home
Monitor
User
Department
Admin
Department
Secure login for authorized officers to manage monitoring, alerts, and incident logs.
Secure Access
Alerts
Logs
Department Login
Enter your credentials to continue.
Username
control
Password
Login


Smart Surveillance
Home
Monitor
Logout
Live
Live Camera Monitoring
Select Area & Video feed. Results will appear in the right panel automatically.
Surveillance Controls
LIVE
Select Area
Panagal Park-T.Nagar-Chennai
Select Video
video1.mp4
Live Feed
Streaming
Detection Result
Auto Refresh
AI DETECTION RESULT
Processing Live Feed


Smart Surveillance
Home
Monitor
Logout
Live
Panagal Park-T.Nagar-Chennai
video1.mp4
Live Feed
Streaming
Detection Result
Auto Refresh
AI DETECTION RESULT
Fight Detected
Processing Live Feed
Fight


Smart Surveillance
Home
Monitor
Logout
Live
Select Area
Panagal Park-T.Nagar-Chennai
Select Video
video3.mp4
Live Feed
Streaming
Detection Result
Auto Refresh
Gun Shoot
AI DETECTION RESULT
Gun Shot Detected
Processing Live Feed


Smart Surveillance
Home
Monitor
Logout
Live
Live Feed
Streaming
Detection Result
Auto Refresh
Fire
AI DETECTION RESULT
Fire Detected
Processing Live Feed


Smart Surveillance
Home
Monitor
User
Department
Admin
Monitor Dashboard
Real-time monitoring dashboard with detection results.
Monitoring
Live CCTV
Result
AI DETECTION RESULT
Abnormal, 15,SS Nagar
Processing Live Feed

CONCLUSION

In conclusion, the comprehensive design and development of the AI-powered surveillance system is grounded in an intelligent, automated approach, utilizing Python and Flask for backend efficiency, Bootstrap for responsive dashboard design, and MySQL for secure and structured data storage. Leveraging advanced AI models such as Swin Transformer for object detection and Pose-GCN for abnormal behavior recognition ensures accurate real-time threat identification, even in crowded or complex environments. The system's key features—including real-time CCTV monitoring, abnormal behavior recognition, threat decision-making with Decision Tree, instant SMS and email alerts, event logging, and a user-friendly monitoring dashboard—collectively contribute to enhancing security management and situational awareness. These modules facilitate proactive threat detection, efficient emergency response, comprehensive incident tracking, and seamless communication among security personnel. With its intelligent functionalities, real-time processing, and scalable architecture, the surveillance system is poised to transform traditional manual monitoring methods, providing a reliable, accurate, and efficient solution for modern security operations in public and private environments.

FUTURE ENHANCEMENT

The project has successfully provided real-time surveillance and threat detection, ensuring improved security and situational awareness. However, there are several areas for future enhancement to further improve coverage, efficiency, and scalability.

- **Drone and Mobile Unit Integration** – Connect with autonomous drones or mobile robots to extend field coverage and capture multiple viewing angles for detected threats.
- **Edge Computing Deployment** – Run AI models on smart cameras or local gateways to reduce latency and maintain real-time detection even during connectivity issues.
- **Emergency Dispatch Integration** – Automate notifications to police, fire, or medical services for rapid response to high-risk threats.
- **Predictive Threat Analytics** – Use historical data to identify patterns and anticipate potential security incidents for proactive intervention.
- **Cloud-Based Centralized Monitoring** – Enable multi-site management with centralized dashboards, real-time collaboration, and remote access to feeds and alerts.

REFERENCE

1. T. Sahoo, B. Mohanty and B. K. Pattanayak, "Moving object detection using deep learning method", *Int. J. Intell. Syst. Appl. Eng.*, vol. 12, no. 9s, pp. 282-290, 2024.
2. Wuyan Liang, Xiaolong Xu and Xiao Fu, "VioNets: efficient multi-modal fusion method based on bidirectional gate recurrent unit and cross-attention graph convolutional network for video violence detection", *Journal of Electronic Imaging*, vol. 32, no. 2, pp. 023031-023031, 2023.
3. Nikita Jain, Vedika Gupta, Usman Tariq and D. Jude Hemanth, "Fast Violence Recognition in Video Surveillance by Integrating Object Detection and Conv-LSTM", International Journal on Artificial Intelligence Tools, vol. 32, no. 03, pp. 2340018, 2023.
4. B. Sauvalle and A. de La Fortelle, "Autoencoder-based background reconstruction and foreground segmentation with background noise estimation", *Proc. IEEE/CVF Winter Conf. Appl. Comput. Vis. (WACV)*, pp. 3243-3254, Jan. 2023.
5. B. N. Subudhi, M. K. Panda, T. Veerakumar, V. Jakhetiya and S. Esakkirajan, "Kernel-induced possibilistic fuzzy associate background subtraction for video scene", *IEEE Trans. Computat. Social Syst.*, vol. 10, no. 3, pp. 1-12, Jan. 2022.
6. M. K. Panda, B. N. Subudhi, T. Bouwmans, V. Jakheytiya and T. Veerakumar, "An end to end encoder–decoder network with multi-scale feature pulling for detecting local changes from video scene", *Proc. 18th IEEE Int. Conf. Adv. Video Signal Based Surveill. (AVSS)*, pp. 1-8, Nov. 2022.
7. P. K. Sahoo, P. Kanungo, S. Mishra and B. P. Mohanty, "Entropy feature and peak-means clustering based slowly moving object detection in head and shoulder video sequences", *J. King Saud Univ.—Comput. Inf. Sci.*, vol. 34, no. 8, pp. 5296-5304, Sep. 2022.
8. L. Fan, T. Zhang and W. Du, "Optical-flow-based framework to boost video object detection performance with object enhancement", *Expert Syst. Appl.*, vol. 170, May 2021.
9. Q. Zhang, T. Xiao, N. Huang, D. Zhang and J. Han, "Revisiting feature fusion for RGB-T salient object detection", *IEEE Trans. Circuits Syst. Video Technol.*, vol. 31, no. 5, pp. 1804-1818, May 2021.

10. M. Mandal, L. K. Kumar, M. Singh Saran and S. K. Vipparthi, "MotionRec: A unified deep framework for moving object recognition", *Proc. IEEE Winter Conf. Appl. Comput. Vis. (WACV)*, pp. 2723-2732, Mar. 2020.
11. Ullah FUM, A Ullah, K Muhammad, IU Haq and SW Baik, "Violence Detection Using Spatiotemporal Features with 3D Convolutional Neural Network", *Sensors (Basel)*, vol. 19, no. 11, pp. 2472, May 2019.
12. A-M. R. Abdali and R. F. Al-Tuma, "Robust Real-Time Violence Detection in Video Using CNN and LSTM", 2019 2nd Scientific Conference of Computer Sciences (SCCS), pp. 104-108, 2019.
13. J. Huang, W. Zou, Z. Zhu and J. Zhu, "An efficient optical flow based motion detection method for non-stationary scenes", *Proc. Chin. Control Decis. Conf. (CCDC)*, pp. 5272-5277, Jun. 2019.
14. P. K. Sahoo, P. Kanungo and S. Mishra, "A fast valley-based segmentation for detection of slowly moving objects", *Signal Image Video Process.*, vol. 12, no. 7, pp. 1265-1272, Oct. 2018.
15. M. F. Savaş, H. Demirel and B. Erkal, "Moving object detection using an adaptive background subtraction method based on block-based structure in dynamic scene", *Optik*, vol. 168, pp. 605-618, Sep. 2018.
16. H. Law and J. Deng, "CornerNet: Detecting objects as paired keypoints", *Proc. Eur. Conf. Comput. Vis. (ECCV)*, pp. 734-750, 2018.
17. P. Kanungo, A. Narayan, P. K. Sahoo and S. Mishra, "Neighborhood based codebook model for moving object segmentation", *Proc. 2nd Int. Conf. Man Mach. Interfacing (MAMI)*, pp. 1-6, Dec. 2017.
18. T.-Y. Lin, P. Goyal, R. Girshick, K. He and P. Dollár, "Focal loss for dense object detection", *Proc. IEEE Int. Conf. Comput. Vis. (ICCV)*, pp. 2999-3007, Oct. 2017.
19. T.-Y. Lin, P. Goyal, R. Girshick, K. He and P. Dollár, "Focal loss for dense object detection", *Proc. IEEE Int. Conf. Comput. Vis. (ICCV)*, pp. 2999-3007, Oct. 2017.
20. J. Redmon, S. Divvala, R. Girshick and A. Farhadi, "You only look once: Unified real-time object detection", Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR), pp. 779-788, Jun. 2016.

- **Python:** Lutz, M., Learning Python, O'Reilly Media.
- **Flask:** Grinberg, M., Flask Web Development: Developing Web Applications with Python, O'Reilly Media.
- **MySQL:** DuBois, P., MySQL, Addison-Wesley.
- **TensorFlow / Keras / OpenCV / Pillow / Matplotlib / NumPy / Pandas / Scikit-learn:** Raschka, S. & Mirjalili, V., Python Machine Learning, Packt Publishing.
- **HTML / CSS / Bootstrap / JavaScript:** Duckett, J., HTML and CSS: Design and Build Websites, Wiley; Duckett, J., JavaScript and jQuery: Interactive Front-End Web Development, Wiley.
- **WampServer:** Khurana, R., Learn WAMP Server for Web Development, BPB Publications.
- **Python Documentation:** https://www.python.org/doc/
- **Flask Documentation:** https://flask.palletsprojects.com/
- **MySQL Documentation:** https://dev.mysql.com/doc/
- **WampServer Official Site:** https://www.wampserver.com
- **TensorFlow Documentation:** https://www.tensorflow.org/
- **Keras Documentation:** https://keras.io/
- **OpenCV Documentation:** https://opencv.org/
- **Pandas Documentation:** https://pandas.pydata.org/
- **NumPy Documentation:** https://numpy.org/
- **Scikit-Learn Documentation:** https://scikit-learn.org/
- **Matplotlib Documentation:** https://matplotlib.org/
- **Pillow Documentation:** https://pillow.readthedocs.io/